

(b) If one is at least 18 years old and it is consistent to assume that one who is physically fit and who passes a test may obtain a pilots license, infer that such a person can obtain a pilots license.

5.5 Show that a Horn clause data base that is consistent is consistent under the CWA assumption. Give an example of a simple Horn clause CWA data base to illustrate consistency.

5.6 For the following database facts, write a completion formula that states that Bill is the only person that lives in Dallas.

LIVESIN(bill,dallas)

LIVESIN(joe,denver)

LIVESIN(sue,phoenix)

OWNS(bill,computer)

STUDENT(sue)

5.7 Determine whether the following modal statements have accessibility relations that are reflexive, transitive, or symmetric:

(a) Bill Brown is alive in the current world.

(b) In the current world and in all worlds in the future of the current world, if Jim Jones is dead in that world, then he will be dead in all worlds in the future of that world.

(c) In the current world or in some world in the future of the current world, John Jones is dead.

5.8 Write modal propositional statements for the following using the operators L and M as described in Section 5.4.

(a) It is necessarily true that the moon is made of green cheese or it is not made of green cheese.

(b) It is possible that if Kennedy were born in Spain Kennedy would speak Spanish.

(c) It is necessarily true that if n is divisible by 4 then n is divisible by 2.

5.9 Show that the dilation of the fuzzy set $\tilde{A} = \text{CON}(\tilde{B})$ is the fuzzy set $\tilde{B}$.

5.10 Give three examples of inferencing with English statements using fuzzy modus ponens (see the example in Section 5.6 under Reasoning with Fuzzy Logic).

5.11 Draw a pictorial definition for the linguistic variable TALL (similar to the variable AGE of Figure 5.8) giving your own subjective values for TALL variables and their values.

5.12 Define a reasonable, real valued fuzzy function for the linguistic variable SHORT (see the function for $\mu_{\text{old}}(x)$).

6

Probabilistic Reasoning

The previous chapter considered methods of representation which extend the expressiveness of classical logic and permit certain types of nonmonotonic reasoning. Representations for vague and imprecise concepts were also introduced with fuzzy set theory and logic. There are other types of uncertainty induced by random phenomena which we have not yet considered. To round out the approaches which are available for commonsense reasoning in AI, we continue in this chapter with theory and methods used to represent probabilistic uncertainties.

6.1 INTRODUCTION

We saw in the previous chapter that a TMS deals with uncertainty by permitting new knowledge to replace old knowledge which is believed to be outdated or erroneous. This is not the same as inferring directly with knowledge that can be given a probability rating based on the amount of uncertainty present. In this chapter, we want to examine methods which use probabilistic representations for all knowledge and which reason by propagating the uncertainties from evidence and assertions to conclusions. As before, the uncertainties can arise from an inability to predict outcomes due to unreliable, vague, incomplete, or inconsistent knowledge.

The probability of an uncertain event A is a measure of the degree of likelihood

of occurrence of that event. The set of all possible events is called the sample space, S . A probability measure is a function $P(\cdot)$ which maps event outcomes $E_1, E_2, \dots$ from S into real numbers and which satisfies the following axioms of probability:

1. $0 \leq P(A) \leq 1$ for any event $A \subseteq S$.
2. $P(S) = 1$, a certain outcome.
3. For $E_i \cap E_j = \emptyset$, for all $i \neq j$ (the E_i are mutually exclusive), $P(E_1 \cup E_2 \cup E_3 \cup \dots) = P(E_1) + P(E_2) + P(E_3) + \dots$

From these three axioms and the rules of set theory, the basic laws of probability can be derived. Of course, the axioms are not sufficient to compute the probability of an outcome. That requires an understanding of the underlying distributions which must be established through one of the following approaches:

1. use of a theoretical argument which accurately characterizes the processes,
2. using one's familiarity and understanding of the basic processes to assign subjective probabilities, or
3. collecting experimental data from which statistical estimates of the underlying distributions can be made.

Since much of the knowledge we deal with is uncertain in nature, a number of our beliefs must be tenuous. Our conclusions are often based on available evidence and past experience, which is often far from complete. The conclusions are, therefore, no more than educated guesses. In a great many situations it is possible to obtain only partial knowledge concerning the possible outcome of some event. But, given that knowledge, one's ability to predict the outcome is certainly better than with no knowledge at all. We manage quite well in drawing plausible conclusions from incomplete knowledge and past experiences.

Probabilistic reasoning is sometimes used when outcomes are unpredictable. For example, when a physician examines a patient, the patient's history, symptoms, and test results provide some, but not conclusive, evidence of possible ailments. This knowledge, together with the physician's experience with previous patients, improves the likelihood of predicting the unknown (disease) event, but there is still much uncertainty in most diagnoses. Likewise, weather forecasters "guess" at tomorrow's weather based on available evidence such as temperature, humidity, barometric pressure, and cloud coverage observations. The physical relationships which govern these phenomena are not fully understood; so predictability is far from certain. Even a business manager must make decisions based on uncertain predictions when the market for a new product is considered. Many interacting factors influence the market, including the target consumer's lifestyle, population growth, potential consumer income, the general economic climate, and many other dependent factors.

In all of the above cases, the level of confidence placed in the hypothesized conclusions is dependent on the availability of reliable knowledge and the experience of the human prognosticator. Our objective in this chapter is to describe some approaches taken in AI systems to deal with reasoning under similar types of uncertain conditions.

6.2 BAYESIAN PROBABILISTIC INFERENCE

The form of probabilistic reasoning described in this section is based on the Bayesian method introduced by the clergyman Thomas Bayes in the eighteenth century. This form of reasoning depends on the use of conditional probabilities of specified events when it is known that other events have occurred. For two events H and E with the probability $P(E) > 0$, the conditional probability of event H , given that event E has occurred, is defined as

$$P(H|E) = P(H \& E) / P(E) \quad (6.1)$$

This expression can be given a frequency interpretation by considering a random experiment which is repeated a large number of times, n . The number of occurrences of the event E , say No. (E), and of the joint event H and E , No. ($H \& E$), are recorded and their relative frequencies rf computed as

$$rf(H \& E) = \frac{\text{No. } (H \& E)}{n} \quad rf(E) = \frac{\text{No. } (E)}{n} \quad (6.2)$$

When n is large, the two expressions (6.2) approach the corresponding probabilities respectively, and the ratio

$$rf(H \& E) / rf(E) = P(H \& E) / P(E)$$

then represents the proportion of times event H occurs relative to the occurrence of E , that is, the approximate conditional occurrence of H with E .

The conditional probability of event E given that event H occurred can likewise be written as

$$P(E|H) = P(H \& E) / P(H) \quad (6.3)$$

Solving 6.3 for $P(H \& E)$ and substituting this in equation 6.1 we obtain one form of Bayes' Rule

$$P(H|E) = P(E|H)P(H) / P(E) \quad (6.4)$$

This equation expresses the notion that the probability of event H occurring when it is known that event E occurred is the same as the probability that E occurs when it is known that H occurred, multiplied by the ratio of the probabilities of the two events H and E occurring. As an example of the use of equation 6.4, consider the problem of determining the probability that a patient has a certain disease $D1$, given that a symptom E was observed. We wish to find $P(D1|E)$.

Suppose now it is known from previous experience that the prior (unconditional) probabilities $P(D1)$ and $P(E)$ for randomly chosen patients are $P(D1) = 0.05$, and $P(E) = 0.15$, respectively. Also, we assume that the conditional probability of the observed symptom given that a patient has disease $D1$ is known from experience to be $P(E|D1) = 0.95$. Then, we easily determine the value of $P(D1|E)$ as

$$P(D1|E) = P(E|D1)P(D1) / P(E) = (0.95 \times 0.05) / 0.15 = 0.32$$

It may be the case that the probability $P(E)$ is difficult to obtain. If that is the case, a different form of Bayes' Rule may be used. To see this, we write equation 6.4 with $\neg H$ substituted in place of H to obtain

$$P(\neg H|E) = \frac{P(E|\neg H)P(\neg H)}{P(E)}$$

Next, we divide equation 6.4 by this result to eliminate $P(E)$ and get

$$\frac{P(H|E)}{P(\neg H|E)} = \frac{P(E|H)P(H)}{P(E|\neg H)P(\neg H)} \quad (6.5)$$

Note that equation 6.5 has two terms that are ratios of a probability of an event to the probability of its negation, $P(H|E) / P(\neg H|E)$ and $P(H) / P(\neg H)$. The ratio of the probability of an event E divided by the probability of its negation is called the *odds* of the event and are denoted as $O(E)$. The remaining ratio $P(E|H) / P(E|\neg H)$ in equation 6.5 is known as the likelihood ratio of E with respect to H . We denote this quantity by $L(E|H)$. Using these two new terms, the odds-likelihood form of Bayes' Rule for equation 6.5 may be written as

$$O(H|E) = L(E|H) \cdot O(H)$$

This form of Bayes' Rule suggests how to compute the posterior odds $O(H|E)$ from the prior odds on H , $O(H)$. That value is proportional to the likelihood $L(E|H)$. When $L(E|H)$ is equal to one, the knowledge that E is true has no effect on the odds of H . Values of $L(E|H)$ less than or greater than one decrease or increase the odds correspondingly. When $L(E|H)$ cannot be computed, estimates may still be made by an expert having some knowledge of H and E . Estimating the ratio rather than the individual probabilities appears to be easier for individuals in such cases. This is sometimes done when developing expert systems where more reliable probabilities are not available.

In the example cited above, $D1$ is either true or false, and $P(D1|E)$ is the interpretation which assigns a measure of confidence that $D1$ is true when it is known that E is true. There is a similarity between E , $P(D1|E)$ and modus ponens discussed in Chapter 4. For example, when E is known to be true and $D1$ and E are known to be related, one concludes the truth of $D1$ with a confidence level $P(D1|E)$.

One might wonder if it would not be simpler to assign probabilities to as

many ground atoms $E_1, E_2, \dots, E_k$ as possible, and compute inferred probabilities (probabilities of $E_j \rightarrow H$ and H) directly from these. The answer is that in general this is not possible. To compute an inferred probability requires a knowledge of the joint distributions of the ground predicates participating in the inference. From the joint distributions, the required marginal distributions may then be computed. The distributions and computations required for that approach are, in general, much more complex than the computations involving the use of Bayes' Rule.

Consider now two events A and $\bar{A}$ which are mutually exclusive ($A \cap \bar{A} = \emptyset$) and exhaustive ($A \cup \bar{A} = S$). The probability of an arbitrary event B can always be expressed as

$$P(B) = P(B \& A) + P(B \& \bar{A}) = P(B|A)P(A) + P(B|\bar{A})P(\bar{A})$$

Using this result, equation 6.4 can be written as

$$P(H|E) = P(E|H)P(H) / [P(E|H)P(H) + P(E|\bar{H})P(\bar{H})] \quad (6.6)$$

Equation 6.6 can be generalized for an arbitrary number of hypotheses H_i , $i = 1, \dots, k$. Thus, suppose the H_i partition the universe; that is, the H_i are mutually exclusive and exhaustive. Then for any evidence E , we have

$$P(E) = \sum_{i=1}^k P(E \& H_i) = \sum_{i=1}^k P(E|H_i)P(H_i)$$

and hence,

$$P(H_i|E) = \frac{P(E|H_i)P(H_i)}{\sum_{j=1}^k P(E|H_j)P(H_j)} \quad (6.7)$$

Finally, to be more realistic and to accommodate multiple sources of evidence $E_1, E_2, \dots, E_m$, we generalize equation 6.7 further to obtain

$$P(H_i|E_1, E_2, \dots, E_m) = \frac{P(E_1, E_2, \dots, E_m|H_i)P(H_i)}{\sum_{j=1}^k P(E_1, E_2, \dots, E_m|H_j)P(H_j)} \quad (6.8)$$

If there are several plausible hypotheses and a number of evidence sources, equation 6.8 can be fairly complex to compute. This is one of the serious drawbacks of the Bayesian approach. A large number of probabilities must be known in advance in order to apply an equation such as 6.8. If there were k hypotheses, H_i , and m sources of evidence, E_j , then $k + m$ prior probabilities must be known in addition to the k likelihood probabilities. The real question then is where does one obtain such a large number of reliable probabilities?

To simplify equation 6.8, it is sometimes assumed that the E_j are statistically independent. In that case, the numerator and denominator probability terms $P(E_1, E_2, \dots, E_m|H_j)$ factor into

$$P(E_1|H_j)P(E_2|H_j) \dots P(E_m|H_j)$$

resulting in a somewhat simpler form. But, even though the computations are straightforward, the number of probabilities required in a moderately large system can still be prohibitive, and one may be forced to simply use subjective probabilities when more reliable values are not available. Furthermore, the E_j are almost never completely independent. Consequently, this assumption may introduce intolerable errors.

The formulas presented above suggest how probabilistic evidence would be combined to produce a likelihood estimate for any given hypothesis. When a number of individual hypotheses are possible and several sources of evidence are available, it may be necessary to compute two or more alternative probabilities and select among them. This may mean that none, one, or possibly more than one of the hypotheses could be chosen. Normally, the one having the largest probability of occurrence would be selected, provided no other values were close. Before accepting such a choice; however, it may be desirable to require that the value exceed some threshold to avoid selecting weakly supported hypotheses. In Section 6.5 we describe a typical system which combines similar values and chooses only those conclusions that exceed a threshold of 0.2.

Bayesian Networks

Network representations of knowledge have been used to graphically exhibit the interdependencies which exist between related pieces of knowledge. Much work has been done in this area to develop a formal syntax and semantics for such representations. We discuss related topics in some detail in Chapter 7 when we consider associative networks and conceptual graphs. Here, however, we are more interested in network representations which depict the degrees of belief of propositions and the causal dependencies that exist between them. Inferencing in a network amounts to propagating the probabilities of given and related information through the network to one or more conclusion nodes.

Network representations for uncertain dependencies are further motivated by observations made earlier. If we wish to represent uncertain knowledge related to a set of propositional variables $x_1, \dots, x_n$ by their joint distribution $P(x_1, \dots, x_n)$, it will require some 2^n entries to store the distribution explicitly. Furthermore, a determination of any of the marginal probabilities x_i requires summing $P(x_1, \dots, x_n)$ over the remaining $n - 1$ variables. Clearly, the time and storage requirements for such computations quickly become impractical. Inferring with such large numbers of probabilities does not appear to model the human process either. On the contrary, humans tend to single out only a few propositions which are known to be causally linked when reasoning with uncertain beliefs. This metaphor leads quite naturally to a form of network representation.

One useful way to portray the problem domain is with a network of nodes which represent propositional variables x_i , connected by arcs which represent causal

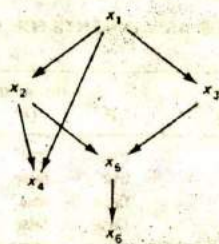


Figure 6.1 Example of Bayesian belief network.

influences or dependencies among the nodes. The strengths of the influences are quantified by conditional probabilities of each variable. For example, to represent causal relationships between the propositional variables $x_1, \dots, x_6$ as illustrated in Figure 6.1, one can write the joint probability $P(x_1, \dots, x_6)$ by inspection as a product of (chain) conditional probabilities

$$P(x_1, \dots, x_6) = P(x_6|x_5)P(x_5|x_2, x_3)P(x_4|x_1, x_2)P(x_3|x_1)P(x_2|x_1)P(x_1)$$

Once such a network is constructed, an inference engine can use it to maintain and propagate beliefs. When new information is received, the effects can be propagated throughout the network until equilibrium probabilities are reached. Pearl (1986, 1987) has proposed simplified methods for updating networks (trees and, more generally, graphs) of this type by fusing and propagating the effects of new evidence and beliefs such that equilibrium is reached in time proportional to the longest path through the network. At equilibrium, all propositions will have consistent probability assignments. Methods for graphs are more difficult. They require the use of dummy variables to transform them to equivalent tree structures which are then easier to work with.

To use the type of probabilistic inference we have been considering, it is first necessary to assign probabilities to all basic facts in the knowledge base. This requires the definition of an appropriate sample space and the assignment of a priori and conditional probabilities. In addition, some method must be chosen to compute the combined probabilities when pooling evidence in a sequence of inference steps (such as Pearl's method). Finally, when the outcome of an inference chain results in one or more proposed conclusions, the alternatives must be compared, and one or more selected on the basis of its likelihood.

6.3 POSSIBLE WORLD REPRESENTATIONS

In Section 5.4 the notion of possible worlds was introduced as a formalism through which an agent could view a set of propositions as being true in some worlds and false in others. We use the possible world concepts in this section to describe a method proposed by Nilsson (1986) which generalizes first order logic in the modelling of uncertain beliefs. The method assigns truth values ranging from 0 to 1 to possible

TABLE 6.1 TRUTH VALUE ASSIGNMENTS FOR THE SET $\{P, P \rightarrow Q, Q\}$

Consistent			Inconsistent		
P	Q	$P \rightarrow Q$	P	Q	$P \rightarrow Q$
true	true	true	true	true	false
true	false	false	true	false	true
false	true	true	false	true	false
false	false	true	false	false	false

worlds. Each set of possible worlds corresponds to a different interpretation of sentences contained in a knowledge base denoted as KB.

Consider first the simple case where a KB contains only the single sentence S . S may be either true or false. We envision S as being true in one set of possible worlds W_1 , and false in another set W_2 . The actual world, the one we are in, must be in one of the two sets, but we are uncertain which one. Our uncertainty is expressed by assigning a probability P to W_1 and $1 - P$ to W_2 . We can say then that the probability of S being true is P .

When our KB contains L sentences, $S_1, \dots, S_L$, more sets of possible worlds are required to represent all consistent truth value assignments. There are 2^L possible truth assignments for L sentences. Some of the assignments may be inconsistent (impossible), however. For example, the set of sentences $\{P, P \rightarrow Q, Q\}$ has 2^3 possible truth assignments, only four of which are consistent as depicted in Table 6.1.

If K of the truth assignments are consistent, K sets of possible worlds $W_1, W_2, \dots, W_K$ are used to model the corresponding interpretations. A probability distribution is then defined over the possible worlds, where P_i is the probability that W_i is the actual world, and $\sum_i P_i = 1$. The probability or belief in any sentence S_j can then be determined by summing the P_i over all W_i in which S_j is true.

These notions can be represented using vector notation. We use the matrix V with L columns and K rows to represent the truth values of the L sentences in each of the K sets W_i . The i^{th} column of V contains a one in row j if sentence S_j is true in W_i and a zero if it is false. Also let the K -dimensional column vector p with components $p_i, i = 1, \dots, K$, represent the possible world probabilities. With this notation, the product of V and p is a vector q which contains the probabilities q_j of sentence S_j , for $j = 1, \dots, L$, that is

$$q = Vp \quad (6.9)$$

The j^{th} component q_j of q is the sum of probabilities of the sets of possible worlds in which S_j is true, or the probabilistic truth value of S_j . As an example of this notation, the matrix equation for the consistent truth value assignments given in Table 6.1 is

$$\mathbf{q} = \begin{bmatrix} 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} p_1 \\ p_2 \\ p_3 \\ p_4 \end{bmatrix}$$

where p_1, p_2, p_3 , and p_4 are the probabilities for the corresponding W_i . Thus, the sentence probabilities are computed as

$$\begin{aligned} q_1 &= p(S_1) = p_1 + p_2 \\ q_2 &= p(S_2) = p_1 + p_3 + p_4 \\ q_3 &= p(S_3) = p_1 + p_3 \end{aligned}$$

Given a KB of sentences with known probabilities (obtained from an expert or other source), we wish to determine the probability of any new sentence S deduced from KB. Alternatively, we may wish to recompute some sentence probabilities in KB if new information has been gained which changes one or more of the original sentences in KB. To compute the probability of S requires that consistent truth values first be determined for S for all sets of possible worlds. A new augmented matrix $\mathbf{V}$ can then be formed by adding a bottom row of ones and zeros to the original $\mathbf{V}$ where the ones and zeros correspond to the truth assignments.

No methods have been developed for the computation of exact solutions for the KB sentence probabilities, although methods for determining approximations were presented for both small and large matrices $\mathbf{V}$. We do not consider those methods here. They may be found in Nilsson (1986). They are based on the use of the probability constraints

$$0 \leq p_i \leq 1, \text{ and } \sum_i p_i = 1.$$

and the fact that consistent probability assignments are bounded by the hyperplanes of a certain convex hull. Suggestions have also been made for the partitioning of larger matrices into smaller ones to simplify the computations.

6.4 DEMPSTER-SHAFER THEORY

As noted in the previous section, the assumption of conditional independence is probably not warranted in many problems. There are other serious drawbacks in using Bayesian theory as a model of uncertain reasoning as well. To begin with, the probabilities are described as a single numeric point value. This can be a distortion of the precision that is actually available for supporting evidence. It amounts to an overstatement of the evidence giving support to many of our beliefs. When we assert with probability 0.7 that the dollar will fall against the Japanese Yen over the next six months, what we really mean is we have a fairly strong conviction there is a chance of about 0.6 to 0.8 say, that it will fall.

Another problem with traditional theory is that there is no way to differentiate

between ignorance and uncertainty. These are distinctly different concepts and should be treated as such. For example, suppose we are informed that one of three terrorist groups, A , B , or C has planted a bomb in a certain government building. We may have some evidence to believe that group C is the guilty one and be willing to assign a measure of this belief equal to $P(C) = 0.8$. On the other hand, without more knowledge of the other two groups, we would not want to say that the probability is 0.1 that each one of them is guilty. Yet, traditional theory would have us distribute an equal amount of the remaining probability to each of the other groups. In fact, we may have no knowledge to justify either the amount of uncertainty nor the equal division of it.

Finally, with classical probability theory, we are forced to regard belief and disbelief as functional opposites. That is, if some proposition A is assigned the probability $P(A) = 0.3$, then we must assign $\neg A$ the probability $P(\neg A) = 0.7$ since we must have $P(A) + P(\neg A) = 1$. This forces us to make an assignment that may be conflicting since it is possible to both believe and disbelieve some propositions by the same amount, making this requirement awkward.

In an attempt to remedy the above problems, a generalized theory has been proposed by Arthur Dempster (1968) and extended by his student Glenn Shafer (1976). It has come to be known as the Dempster-Shafer theory of evidence. The theory is based on the notion that separate probability masses may be assigned to all subsets of a universe of discourse rather than just to indivisible single members as required in traditional probability theory. As such, it permits the inequality $P(A) + P(\neg A) \leq 1$.

In the Dempster-Shafer theory, we assume a universe of discourse U and a set corresponding to n propositions, exactly one of which is true. The propositions are assumed to be exhaustive and mutually exclusive. Let 2^U denote all subsets of U including the empty set and U itself (there are 2^n such subsets). Let the set function m (sometimes called a basic probability assignment) defined on 2^U , be a mapping to $[0,1]$,

$$\begin{aligned} m: 2^U &\rightarrow [0,1], \text{ be such that for all subsets } A \subseteq U \\ m(\emptyset) &= 0 \\ \sum_{A \subseteq U} m(A) &= 1 \end{aligned}$$

The function m defines a probability distribution on 2^U (not just on the singletons of U as in classical theory). It represents the measure of belief committed exactly to A . In other words, it is possible to assign belief to each subset A of U without assigning any to anything smaller.

A belief function, Bel , corresponding to a specific m for the set A , is defined as the sum of beliefs committed to every subset of A by m . That is, $\text{Bel}(A)$ is a measure of the total support or belief committed to the set A and sets a minimum value for its likelihood. It is defined in terms of all belief assigned to A as well as to all proper subsets of A . Thus,

$$\text{Bel}(A) = \sum_{B \subset A} m(B)$$

For example, if U contains the mutually exclusive subsets A , B , C , and D then

$$\begin{aligned} \text{Bel}(\{A, C, D\}) = & m(\{A, C, D\}) + m(\{A, C\}) + m(\{A, D\}) + m(\{C, D\}) \\ & + m(\{A\}) + m(\{C\}) + m(\{D\}) \end{aligned}$$

In Dempster-Shafer theory, a belief interval can also be defined for a subset A . It is represented as the subinterval $[\text{Bel}(A), P1(A)]$ of $[0, 1]$. $\text{Bel}(A)$ is also called the *support* of A , and $P1(A) = 1 - \text{Bel}(\bar{A})$, the *plausibility* of A .

We define $\text{Bel}(\emptyset) = 0$ to signify that no belief should be assigned to the empty set and $\text{Bel}(U) = 1$ to show that the truth is contained within U . The subsets A of U are called the *focal elements* of the support function Bel when $m(A) > 0$.

Since $\text{Bel}(A)$ only partially describes the beliefs about proposition A , it is useful to also have a measure of the extent one believes in $\bar{A}$, that is, the *doubts* regarding A . For this, we define the doubt of A as $D(A) = \text{Bel}(\bar{A})$. From this definition it will be seen that the upper bound of the belief interval noted above, $P1(A)$, can be expressed as $P1(A) = 1 - D(A) = 1 - \text{Bel}(\bar{A})$. $P1(A)$ represents an upper belief limit on the proposition A . The belief interval, $[\text{Bel}(A), P1(A)]$, is also sometimes referred to as the *confidence* in A , while the quantity $P1(A) - \text{Bel}(A)$ is referred to as the *uncertainty* in A . It can be shown that (Prade 1983)

$$P1(\emptyset) = 0, P1(U) = 1$$

For all A ,

$$\begin{aligned} P1(A) &\geq \text{Bel}(A), \\ \text{Bel}(A) + \text{Bel}(\bar{A}) &\leq 1, \\ P1(A) + P1(\bar{A}) &\geq 1, \text{ and} \end{aligned}$$

For $A \subseteq B$,

$$\text{Bel}(A) \leq \text{Bel}(B), P1(A) \leq P1(B)$$

In interpreting the above definitions, it should be noted that a portion of belief may be committed to a set of propositions, but need not be, and if committed, it is not necessary to commit any belief to its negation. However, a belief committed to a proposition is committed to any other proposition it implies.

A few specific interval belief values will help to clarify the intended semantics. For example,

- [0,1] represents no belief in support of the proposition
- [0,0] represents the belief the proposition is false
- [1,1] represents the belief the proposition is true
- [.3,1] represents partial belief in the proposition
- [0,.8] represents partial disbelief in the proposition
- [.2,.7] represents belief from evidence both for and against the proposition

When evidence is available from two or more independent knowledge sources Bel_1 and Bel_2 , one would like to pool the evidence to reduce the uncertainty. For this, Dempster has provided such a combining function denoted as $Bel_1 \circ Bel_2$.

Given two basic probability assignment functions, m_1 and m_2 corresponding to the belief functions Bel_1 and Bel_2 , let $A_1, \dots, A_k$ be the focal elements for Bel_1 and $B_1, \dots, B_p$ be the focal elements for Bel_2 . Then $m_1(A_i)$ and $m_2(B_j)$ each assign probability masses on the unit interval. They can be orthogonally combined as depicted with the square illustrated in Figure 6.2 (Garvey et al., 1981).

The unit square in Figure 6.2 represents the total probability mass assigned by both m_1 and m_2 for all of their common subsets. A particular subrectangle within the square, shown as the intersection of the sets A_i and B_j , has committed to it the measure $m_1(A_i)m_2(B_j)$. Likewise, any subset C of U may have one, or more than one, of these rectangles committed to it. Therefore, the total probability mass committed to C will be

$$\sum_{A_i \cap B_j = C} m_1(A_i)m_2(B_j) \quad (6.9)$$

where the summation is over all i and j .

The sum in equation 6.9 must be normalized to account for the fact that some intersections $A_i \cap B_j = \emptyset$ will have positive probability which must be discarded. The final form of Dempster's rule of combination is then given by

$$m_1 \circ m_2 = \frac{\sum_{A_i \cap B_j} m_1(A_i)m_2(B_j)}{\sum_{A_i \cap B_j \neq \emptyset} m_1(A_i)m_2(B_j)} \quad (6.10)$$

where the summations are taken over all i and j .

As an example of the above concepts, recall once again the problem of identifying the terrorist group or groups responsible for a certain attack in some country. Suppose any of four known terrorist organizations A, B, C , and D could have been responsible for the attack. The possible subsets of U in this case form a lattice of sixteen subsets (Figure 6.3).

Assume one piece of evidence supports the belief that groups A and C were

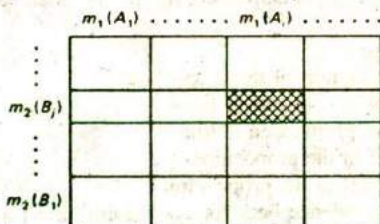
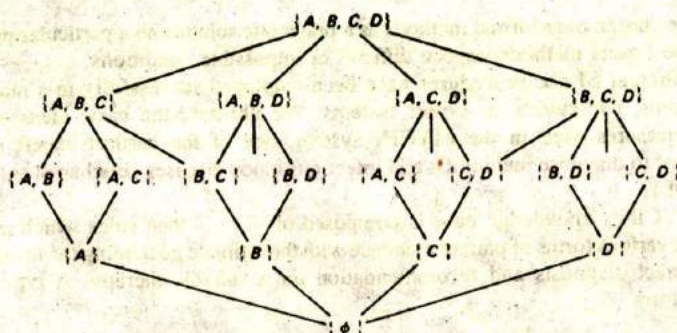


Figure 6.2 Composition of probability mass from sources Bel_1 and Bel_2 .

Figure 6.3 Lattice of subsets of the universe U .

responsible to a degree of $m_1(\{A, C\}) = 0.6$, and another source of evidence disproves the belief that C was involved (and therefore supports the belief that the three organizations, A , B , and D were responsible; that is $m_2(\{A, B, D\}) = 0.7$. To obtain the pooled evidence, we compute the following quantities (summarized in Table 6.2).

$$\begin{aligned}
 m_1 \circ m_2(\{A\}) &= (0.6) * (0.7) = 0.42 \\
 m_1 \circ m_2(\{A, C\}) &= (0.6) * (0.3) = 0.18 \\
 m_1 \circ m_2(\{A, B, D\}) &= (0.4) * (0.7) = 0.28 \\
 m_1 \circ m_2(\{U\}) &= (0.4) * (0.3) = 0.12 \\
 m_1 \circ m_2 &= 0 \text{ for all other subsets of } U \\
 \text{Bel}_1(\{A, C\}) &= m(\{A, C\}) + m(\{A\}) + m(\{C\})
 \end{aligned}$$

TABLE 6.2 TABLEAU OF COMBINED VALUES OF BELIEF FOR m_1 AND m_2

		m_2	
		$\{A, B, D\}$ (0.7)	U (0.3)
m_1	$\{A, C\}$ (0.6)	$\{A\}$ (0.42)	$\{A, C\}$ (0.18)
	U (0.4)	$\{A, B, D\}$ (0.28)	U (0.12)

6.5 AD HOC METHODS

The so-called ad hoc methods of dealing with uncertainty are methods which have no formal theoretical basis (although they are usually patterned after probabilistic concepts). These methods typically have an intuitive, if not a theoretical, appeal.

They are chosen over formal methods as a pragmatic solution to a particular problem, when the formal methods impose difficult or impossible conditions.

Different ad hoc procedures have been employed successfully in a number of AI systems, particularly in expert systems. We illustrate the basic ideas with the belief measures used in the MYCIN system, one of the earliest expert systems developed to diagnose meningitis and infectious blood diseases (Buchanan and Shortliffe, 1984).

MYCIN's knowledge base is composed of if . . . then rules which are used to assess various forms of patient evidence with the ultimate goal being the formulation of a correct diagnosis and recommendation for a suitable therapy. A typical rule has the form

- IF: The stain of the organism is gram positive, and
 The morphology of the organism is coccus, and
 The growth conformation of the organism is chains
 THEN: There is suggestive evidence (0.7) that the identity of the organism is streptococcus

This is a rule that would be used by the inference mechanism to help identify the offending organism. The three conditions given in the IF part of the rule refer to attributes that help to characterize and identify organisms (the stain, morphology, and growth conformation). When such an identification is relatively certain, an appropriate therapy may then be recommended.

The numeric value (0.7) given in the THEN part of the above rule corresponds to an expert's estimate of degree of belief one can place in the rule conclusion when the three conditions in the IF part have been satisfied. Thus, the belief associated with the rule may be thought of as a (subjective) conditional probability $P(H|E_1, E_2, E_3) = 0.7$, where H is the hypothesis that the organism is streptococcus, and E_1 , E_2 , and E_3 correspond to the three pieces of joint evidence given in the IF part, respectively.

MYCIN uses measures of both belief and disbelief to represent degrees of confirmation and disconfirmation respectively in a given hypothesis. The basic measure of belief, denoted by $MB(H, E)$, is actually a measure of the *increased* belief in hypothesis H due to the evidence E . This is roughly equivalent to the estimated increase in probability of $P(H|E)$ over $P(H)$ given by an expert as a result of the knowledge gained by E . A value of 0 corresponds to no increase in belief and 1 corresponds to maximum increase or absolute belief. Likewise, $MD(H, E)$ is a measure of the increased disbelief in hypothesis H due to evidence E . MD ranges from 0 to +1, also, with +1 representing maximum increase in disbelief, (total disbelief) and 0 representing no increase. In both measures, the evidence E may be absent or may be replaced with another hypothesis, $MB(H_1, H_2)$. This represents the increased belief in H_1 given H_2 is true.

In an attempt to formalize the uncertainty measure in MYCIN, definitions of MB and MD have been given in terms of prior and conditional probabilities. It

should be remembered, however, the actual values are often subjective probability estimates provided by a physician. We have for the definitions

$$MB(H,E) = \begin{cases} 1 & \text{if } P(H) = 1 \\ \frac{\max[P(H|E), P(H)] - P(H)}{\max[1, 0] - P(H)} & \text{otherwise} \end{cases} \quad (6.11)$$

$$MD(H,E) = \begin{cases} 1 & \text{if } P(H) = 0 \\ \frac{\min[P(H|E), P(H)] - P(H)}{\min[1, 0] - P(H)} & \text{otherwise} \end{cases} \quad (6.12)$$

Note that when $0 < P(H) < 1$, and E and H are independent (so $P(H|E) = P(H)$), then $MB = MD = 0$. This would be the case if E provided no useful information.

The two measures MB and MD are combined into a single measure called the certainty factor (CF), defined by

$$CF(H,E) = MB(H,E) - MD(H,E) \quad (6.13)$$

Note that the value of CF ranges from -1 (certain disbelief) to $+1$ (certain belief). Furthermore, a value of $CF = 0$ will result if E neither confirms nor unconfirms H (E and H are independent).

In MYCIN, each rule hypothesis H_i has an associated MB and MD initially set to zero. As evidence is accumulated, they are updated using intermediate combining functions, and, when all applicable rules have been executed, a final CF is calculated for each H_i . These are then compared and the largest cumulative confirmations or disconfirmations are used to determine the appropriate therapy. A threshold value of $|CF| > 0.2$ is used to prevent the acceptance of a weakly supported hypothesis.

In the initial assignment of belief values an expert will consider all available confirming and disconfirming evidence, $E_1, \dots, E_k$, and assign appropriate, consistent values to both. For example, in the assignment process, a value of 1 should be made if and only if a piece of evidence logically implies H ($\neg H$) with certainty. Additional rules related to the assignment process must also be carefully followed when using such methods.

Ad hoc methods have been used in a large number of knowledge-based systems, more so than have the more formal methods. This is largely because of the difficulties encountered in acquiring large numbers of reliable probabilities related to the given domain and to the complexities of the ensuing calculations. But, in bypassing the formal approaches one should question what end results can be expected. Are they poorer than would be obtained using formal methods? The answer to this question seems to be not likely. Sensitivity analyses (Buchanan et al., 1984) seem to indicate that the outcomes are not too sensitive to either the method nor the actual values used for many systems. However, much work remains to be done in this area before a useful theory can be formulated.

6.6 HEURISTIC REASONING METHODS

The approaches to uncertainty described so far seem to lack an AI flavor. The uncertainty in a given hypothesis is represented as a number which is combined with and compared to other numbers until a final number is translated into a weighted conclusion. Once the uncertainties have been translated into numbers, the cause of the uncertainty, its relative importance, necessity, and other factors are lost. Furthermore, this does not appear to be the process with which humans reason about uncertainty. Although we do weigh evidence both pro and con, we usually do not arrive at a net numeric estimate of a conclusion, only whether or not the conclusion is justified. In place of numbers, our reasoning appears to depend more on heuristics when reasoning with uncertain, incomplete knowledge. In this section we briefly consider this type of approach.

Heuristic methods are based on the use of procedures, rules, and other forms of encoded knowledge to achieve specified goals under uncertainty. Using both domain specific and general heuristics, one of several alternative conclusions may be chosen through the strength of positive versus negative evidence presented in the form of justifications or endorsements. The endorsement weights employed in such systems need not be numeric. Some form of ordering or preference selection scheme must be used, however.

For example, in a prototype system (named SOLOMON) developed by Paul Cohen (1985), endorsements in the form of domain and general heuristics are used to reason about uncertainties associated with a client's investment portfolio. In selecting investments for clients, there are many sources of uncertainty to contend with. First, there is uncertainty related to the client's lifestyle and financial status as well as his or her objectives. Secondly, there are varying degrees of uncertainty related to all types of investments. Some important factors which influence an investor's status include age, preretirement income expected, net worth, income needs, retirement programs, and tax bracket. Factors related to investments include corporate earnings, dividends, bond and money market yields, the direction of the market, and the rate of inflation, to name a few.

Endorsements in the SOLOMON system are justifications or reasons for believing or disbelieving a proposition or an inference. They are provided by experts as heuristics in place of numeric probability estimates. They are expressed as rules about the properties and relationships of domain objects. For example, heuristics used to reason about the uncertainties related to a client's status might take the form of rules such as

- IF: Client income need is high and net worth is medium to high,
THEN: Risk-tolerance level is medium.
- IF: Client tax bracket is high and risk-tolerance level is low,
THEN: Tax-exempt mutual-funds are indicated.
- IF: Client age is high and income needs are high and retirement income is medium,

THEN: Risk-tolerance is low.

IF: Two positive endorsements are medium or high and one negative endorsement is high.

THEN: Then favor the positive choice.

Endorsements are used to control the reasoning process in at least two different ways. First, preference is given to rules which are strongly supported. Second, endorsements permit the condition or left-hand side of a rule to be satisfied (or rejected) without finding an exact match in the KB. The SOLOMON system is goal driven and uses a form of backward rule chaining. A goal is achieved when all of the inference conditions in the left-hand side of the goal rule have been proved. This requires proving subgoals and sub-subgoals until the chain of inferences is completed.

The control structure in SOLOMON is based on the use of an agenda where tasks derived from rules are ordered for completion on the strength of their endorsements. Strongly endorsed tasks are scheduled ahead of weakly endorsed ones. And when a task is removed for execution, endorsements are checked to see if they are still worth completing.

Endorsements are propagated over inferences $P \rightarrow Q$ by combining, replacing, or dropping endorsements E_P associated with antecedents P , endorsements of the implication itself, and other evidential relationships between Q and conclusions in the KB.

The SOLOMON system borrowed several design features from another heuristic reasoning system developed by Douglas Lenat called AM (Davis and Lenat, 1982). AM discovers basic concepts in mathematics by investigating examples of a newly generated conjecture and looking for regularities and extreme or boundary values in the examples. With an exponential number of available tasks, the system is always uncertain about what to work on next. For this, AM also uses an agenda to schedule its tasks for further investigation. Here again, heuristics are used to control the reasoning process. The system does this by developing a numerical "interest" factor rating for tasks which is used to determine the task's position on the agenda. Like the SOLOMON system, AM gives more strength to rules which have supportive evidence.

Although both AM and SOLOMON take into account the importance of the evidence, SOLOMON differs in one respect. SOLOMON also accounts for the accuracy of the evidence just as the testimony of an eyewitness is more convincing than circumstantial evidence. AM is unable to assess accuracy as such.

6.7 SUMMARY

Commonsense reasoning is nonmonotonic in nature. It requires the ability to reason with incomplete, and uncertain knowledge. Many AI systems model uncertainty with probabilities or levels of confidence and probability inference computations.

Each belief is assigned a degree of truthfulness or plausibility. Evidence is pooled through combining functions which compute a combined belief probability. Two popular approaches were considered in this chapter: the Bayesian probability method and the Dempster-Shafer approach.

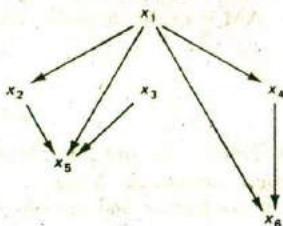
The Bayesian approach depends on the use of known-prior and likely probabilities to compute conditional probabilities. The Dempster-Shafer approach on the other hand is a generalization of classical probability theory which permits the assignment of probability masses (beliefs) to all subsets of the universe and not just to the basic elements. A measure of belief for an assertion is then computed as a subinterval of $[0,1]$, where the length of the interval measures the uncertainty.

In addition to methods based on formal theories the more pragmatic ad hoc approaches to uncertain reasoning were examined. In particular, the procedures used in MYCIN which combine measures of belief and disbelief into certainty factors were described. Because of its success, a number of expert systems designs have been patterned after this method.

Finally, heuristic, nonnumeric approaches to the uncertainty problem were considered. Here, endorsements for a given alternative would outweigh negative factors if general rules, data, and other domain knowledge provided stronger support. The SOLOMON and AM systems use a form of heuristic control to reason with uncertain knowledge.

EXERCISES

- 6.1 Find the probability of the event A when it is known that some event B occurred. From experiments it has been determined that $P(B|A) = 0.84$, $P(A) = 0.2$, and $P(B) = 0.34$.
- 6.2 Prove that if A and B are independent, $P(A|B) = P(A)$. (Note that A and B are independent if and only if $P(A \& B) = P(A)P(B)$).
- 6.3 From basic set theory prove that $P(\bar{A}) = 1 - P(A)$, and that $P(\bar{B}|A) = 1 - P(B|A)$.
- 6.4 Is it possible to compute $P(A|\bar{B})$ when you are only given $P(A)$, $P(B|A)$, and $P(B)$? Explain your answer.
- 6.5 Write the joint distribution of x_1, x_2, x_3, x_4, x_5 , and x_6 as a product of the chain conditional probabilities for the following causal network:



- 6.6 Define the sentences S_1 , S_2 , and S_3 as $S_1 = P$, $S_2 = Q$, and $S_3 = P \rightarrow Q$. Determine the probabilistic truth values of S_1 , S_2 , and S_3 when it is known that the probabilities of the possible worlds are given by $P(W_1) = 1/4$, $P(W_2) = 1/8$, $P(W_3) = 1/8$, and $P(W_4) = 1/2$.
- 6.7 Write a LISP program that computes certainty factors for rules based on the ad hoc MYCIN model.
- 6.8 Dempster-Shafer computations were given for four terrorist organizations A , B , C , and D in Section 6.4. Suppose now that new evidence (m_3) indicates that organization C was indeed responsible to a degree of 0.8. This requires that values for $m_3 \circ m_4$ be computed, where $m_4 = m_1 + m_2$. Compute a new intersection tableau for the new evidence, that is compute $m_3(C)$ and $m_3(U)$ versus $m_4(A)$, $m_4(CA)$, $m_4(ABD)$, and $m_4(U)$.
- 6.9 In what ways do endorsement justifications differ from probabilistic justifications?
- 6.10 In what ways do endorsement justifications differ from fuzzy logic justifications?

Structured Knowledge: Graphs, Frames, and Related Structures

The representations considered up to this point focused primarily on expressiveness, validity, consistency, inference methods, and related topics. Little consideration was given to the way in which the knowledge was structured and how it might be viewed by designers, or to the type of data structures that should be used internally. Neither was much consideration given to effective methods of organizing the knowledge structures in memory. In this, and the following two chapters, we address such problems.

In this chapter, we first look at associative networks as a form of representation and see examples of their use and flexibility. Next, we look at frames, a generalized structure recently introduced as a method of grouping and linking related chunks of knowledge. Finally, we consider structures closely related to frames known as scripts and memory organization packets. We compare them with the frame structures and see how they are used in reasoning systems.

7.1 INTRODUCTION

The representations studied in Chapter 4 are suitable for the expression of fairly simple facts. They can be written in clausal form as independent units and placed in a KB in any order. Inference is straightforward with a procedure such as chaining

```
PROFESSION(bob,professor)
FACULTY(bob,engineering)

MARRIED(bob,sandy)
FATHER-OF(bob,sue,joe)
DRIVES(bob,buick)
OWNS(bob,house)
MARRIED(x,y) V MARRIED(y,x)
```

Figure 7.1 Facts in a KB given in clausal form.

or resolution. For example, facts about Bob, a university professor, might be entered as clauses in a KB as depicted in Figure 7.1.

The entries in the KB of Figure 7.1 have no particular order or grouping associated with them. Furthermore, in representing various facts about Bob, it was necessary to repeat Bob's name for each association given. All facts appear independently, without any linkage to other facts, even though they may be closely related conceptually (Bob is married, owns a house, has children, drives a Buick, and so forth).

For small KBs, the representation used in Figure 7.1 presents no problem. Adding, or otherwise changing facts in the KB is easy enough, and a search of all clauses in the KB can be made if necessary when performing inferences. When the quantity of information becomes large and more complex, however, the acquisition, comprehension, use, and maintenance of the knowledge can become difficult or even intractable. In such cases, some form of knowledge structuring and organization becomes a necessity.

Real-world problem domains typically involve a number and variety of different objects interacting with each other in different ways. The objects themselves may require extensive characterizations, and their interaction relationships with other objects may be very complex.

7.2 ASSOCIATIVE NETWORKS

Network representations provide a means of structuring and exhibiting the structure in knowledge. In a network, pieces of knowledge are clustered together into coherent semantic groups. Networks also provide a more natural way to map to and from natural language than do other representation schemes. Network representations give a pictorial presentation of objects, their attributes and the relationships that exist between them and other entities. In this section, we describe general associative

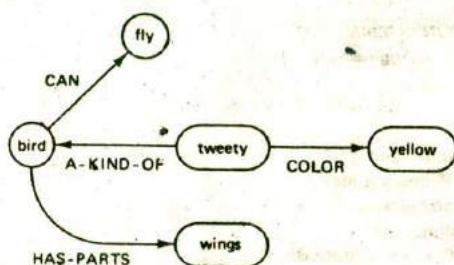


Figure 7.2 Fragment of an associative network.

networks (also known as semantic networks) and conceptual graphs and give some of their properties.

Associative networks are directed graphs with labeled nodes and arcs or arrows. The language used in constructing a network is based on selected domain primitives for objects and relations as well as some general primitives. A fragment of a simple network is illustrated in Figure 7.2. In the figure, a class of objects known as Bird is depicted. The class has some properties and a specific member of the class named Tweety is shown. The color of Tweety is seen to be yellow.

Associative networks were introduced by Quillian (1968) to model the semantics of English sentences and words. He called his structures semantic networks to signify their intended use. He developed a system which "found" the meanings between words by the paths connecting them. The connections were determined through a kind of "spreading activation" between the two words.

Quillian's model of semantic networks has a certain intuitive appeal in that related information is clustered and bound together through relational links. The knowledge required for the performance of some task is typically contained within a narrow domain or "semantic vicinity" of the task. This type of organization in some way, resembles the way knowledge is stored and retrieved in humans.

The graphical portrayal of knowledge can also be somewhat more expressive than other representation schemes. This probably accounts for the popularity and the diversity of representation models for which they have been employed. They have, for example, been used in a variety of systems such as, natural language understanding, information retrieval, deductive data bases, learning systems, computer vision, and in speech generation systems.

Syntax and Semantics of Associative Networks

Unlike FOPL, there is no generally accepted syntax nor semantics for associative networks. Such rules tend to be designer dependent and vary greatly from one implementation to another. Most network systems are based on PL or FOPL with extensions, however. The syntax for any given system is determined by the object and relation primitives chosen and by any special rules used to connect nodes. Some efforts have been made toward the establishment of acceptable standards by

Schubert, Goebel, and Cercone (1979), Shapiro (1979), Hendrix (1979), and Brachman (1979). Later in this section we will review one formal approach to graphical representations which was recently proposed by John Sowa (1984).

Basically, the language of associative networks is formed from letters of the alphabet, both upper- and lowercase, relational symbols, set membership and subset symbols, decimal digits, square and oval nodes, and directed arcs of arbitrary length. The word symbols used are those which represent object constants and n -ary relation constants. Nodes are commonly used for objects or nouns, and arcs (or arc nodes) for relations. The direction of an arc is usually taken from the first to subsequent arguments as they appear in a relational statement. Thus, OWNS(bobs,house) would be written as

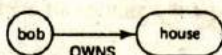


Figure 7.3 depicts graphically some additional concepts not expressed in Figure 7.1.

A number of arc relations have become common among users. They include such predicates as ISA, MEMBER-OF, SUBSET-OF, AKO (a-kind-of), HAS-

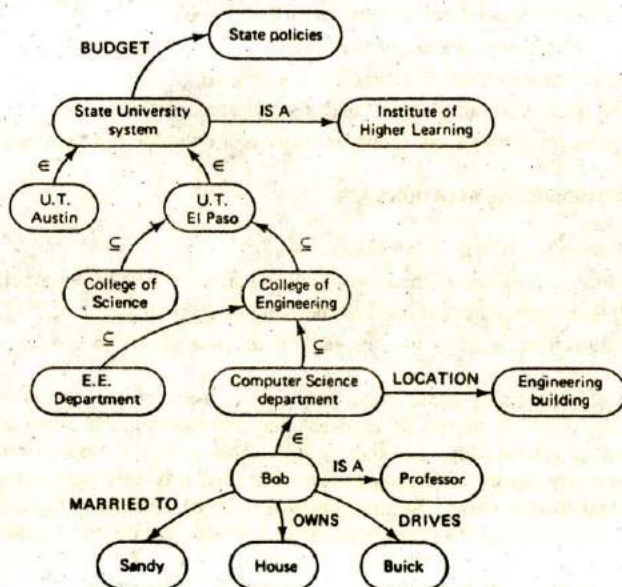


Figure 7.3 Associative network node and arc types.

PARTS, INSTANCE-OF, AGENT, ATTRIBUTES, SHAPED-LIKE, and so forth. Less common arcs have also been used to express modality relations (time, manner, mood), linguistics case relations (theme, source, goal), logical connectives (or, not, and, implies), quantifiers (all, some), set relations (superset, subset, member), attributes, and quantification (ordinal, count).

One particular arc or link, the ISA (is a) link, has taken on a special meaning. It signifies that Bob is a professor and that the state university system is an institute of higher learning. ISA relationships occur in many representations of worlds: Bill is a student, a cat is a furry animal, a tree is a plant, and so on. The ISA link is most often used to represent the fact that an object is of a certain type (predication) or to express the fact that one type is a subtype of another (for example, conditional quantification).

Brachman (1979, 1983) has given an interesting description of the background and uses of this now famous link. For example, the ISA predicate has been used to exhibit the following types of structures:

GENERIC-GENERIC RELATIONSHIPS

Subset-Superset (fighting ships-battleships)

Generalization-Specialization (restaurant-fast-foods)

AKO (an elephant is a kind of mammal)

Conceptual containment (a triangle is a polygon)

Sets and their type (an elephant and a set of elephants)

Role value restrictions (an elephant trunk is a cylinder 1.3 meters in length)

GENERIC-INDIVIDUAL RELATIONSHIPS

Set membership (Clyde is a camel)

Predication (predicate application to individual as in BROWN(camel))

Conceptual containment (king and the King of England)

Abstraction (the "eagle" in "the eagle is an endangered species")

Figure 7.3 illustrates some important features associative networks are good at representing. First, it should be apparent that networks clearly show an entity's attributes and its relationships to other entities. This makes it easy to retrieve the properties an entity shares with other entities. For this, it is only necessary to check direct links tied to that entity. Second, networks can be constructed to exhibit any hierarchical or taxonomic structure inherent in a group of entities or concepts. For example, at the top of the structure in Figure 7.3 is the Texas State University System. One level down from this node are specific state universities within the system. One of these universities, the University of Texas at El Paso, is shown with some of its subparts, colleges, which in turn have subparts, the different departments. One member of the Computer Science Department is Bob, a professor who

owns a house and is married to Sandy. Finally, we see that networks depict the way in which knowledge is distributed or clustered about entities in a KB.

Associative network structures permit the implementation of property inheritance, a form of inference. Nodes which are members or subsets of other nodes may inherit properties from their higher level ancestor nodes. For example, from the network of Figure 7.4, it is possible to infer that a mouse has hair and drinks milk.

Property inheritance of this type is recognized as a form of default reasoning. The assumption is made that unless there is information to the contrary, it is reasonable for an entity to inherit characteristics from its ancestor nodes. As the name suggests, this type of inheritance is called default inheritance. When an object does not or cannot inherit certain properties, it would be assigned values of its own which override any inherited ones.

Data structures patterned after associative networks also permit the efficient storage of information since it is only necessary to explicitly store objects and shared properties once. Shared properties are attached only to the highest node in a structure to which they apply. For example, in LISP, Bob's associations (Figure 7.3) can be implemented with property lists where all of Bob's properties are linked to one atom.

```
(putprop 'bob 'cs-dept 'member-of)
(putprop 'bob 'professor 'isa)
(putprop 'bob 'sandy 'married-to)
(putprop 'bob 'house 'owns)
(putprop 'bob 'buick 'drives)
```

The semantics of associative networks are sometimes defined along the same lines as that of traditional logics. In fact, some network system definitions provide a means of mapping to and from PL or FOPL expressions. For these systems, the semantics are based on interpretations. Thus, an interpretation satisfies a portion of a network if and only if all arc relations hold in the given portion.

Inference procedures for networks can also parallel those of PL and FOPL. If a class *A* of objects has some property *P*, and *a* is a member of *A*, we infer that *a* has property *P*. Syntactic inference in networks can also be defined using parallels to traditional logics such as unification, chaining, modus ponens, and even resolution.

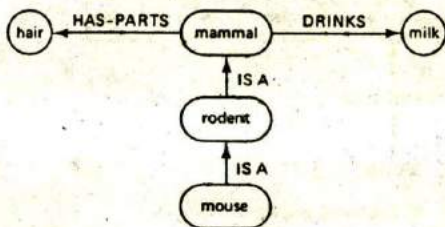


Figure 7.4 Property inheritance in a hierarchical network.

These procedures are implemented through node and arc matching processes and operators which insert, erase, copy, simplify, and join networks. We examine some typical inferencing procedures in more detail below.

Conceptual Graphs

Although there are no commonly accepted standards for a syntax and semantics for associative networks, we present an approach in this section which we feel may at least become a de facto standard in the future. It is based on the use of the conceptual graph as a primitive building block for associative networks. The formalism of these graphs has been adopted as a basic representation by a number of AI researchers and a variety of implementations using conceptual graphs are currently under development. Much of the popularity of these graphs has been due to recent work by John Sowa (1984) and his colleagues.

A conceptual graph is a graphical portrayal of a mental perception which consists of basic or primitive concepts and the relationships that exist between the concepts. A single conceptual graph is roughly equivalent to a graphical diagram of a natural language sentence where the words are depicted as concepts and relationships. Conceptual graphs may be regarded as formal building blocks for associative networks which, when linked together in a coherent way, form a more complex knowledge structure. An example of such a graph which represents the sentence "Joe is eating soup with a spoon" is depicted in Figure 7.5.

In Figure 7.5, concepts are enclosed in boxes and relations between the concepts are enclosed in ovals. The direction of the arrow corresponds to the order of the arguments in the relation they connect. The last or n th arc (argument) points away from the circle relation and all other arcs point toward the relation.

Concept symbols refer to entities, actions, properties, or events in the world. A concept may be individual or generic. Individual concepts have a type field followed by a referent field. The concept [PERSON;joe] has type PERSON and referent Joe. Referents like joe and food in Figure 7.5 are called individual concepts since they refer to specific entities. EAT and SPOON have no referent fields since they are generic concepts which refer to unspecified entities. Concepts like AGENT, OBJECT, INSTRUMENT, and PART are obtained from a collection of standard concepts. New concepts and relations can also be defined from these basic ones.

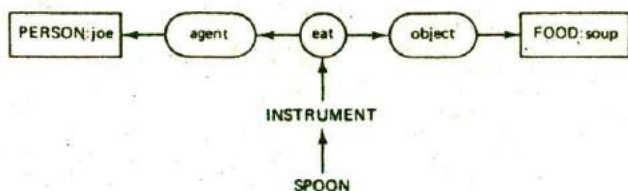


Figure 7.5 A conceptual graph.

A linear conceptual graph form which is easier to present as text can also be given. The linear form equivalent to the above sentence is

$$\begin{aligned} &[\text{PERSON:joel}] \leftarrow (\text{AGENT}) \leftarrow [\text{EAT}] - \\ &(\text{OBJECT}) \rightarrow [\text{FOOD:soup}] \\ &(\text{INSTRUMENT}) \rightarrow [\text{SPOON}] \end{aligned}$$

where square brackets have replaced concept boxes and parentheses have replaced relation circles.

The language of conceptual graphs is formed from upper- and lowercase letters of the alphabet, boxes and circles, directed arcs, and a number of special characters including -, ?, !, *, #, @, V, ~, ", ., [,], (,), →, ←, {, and }. Some symbols are used to exhibit the structure of the graph, while others are used to determine the referents.

The dash signifies continuation of the linear graph on the next line. The question mark is used to signify a query about a concept when placed in the referent field: [HOUSE:?] means which house? The exclamation mark is used for emphasis to draw attention to a concept. The asterisk signifies a variable or unspecified object: [HOUSE:*x] means a house or some house. The pound sign signifies a definite article known to the speaker. For example, [House:#432] refers to a specific house, house number 432. The @ symbol relates to quantification: [HOUSE:@n] means n houses. V signifies every or all, the same as in FOPL. The tilde is negation. Double quotation marks delimit literal strings. And the colon, brackets, parentheses, and directed arcs are used to construct graph structures as illustrated above.

Since conceptual graphs and FOPL are both a form of logical system, one might expect that it is possible to map from one representation to the other. Indeed this is the case, although some mappings will, in general, result in second order FOPL statements.

To transform a conceptual graph to a predicate logic statement requires that unique variable names be assigned to every generic concept of the graph. Thus, the concepts EAT and FOOD of Figure 7.5 would be assigned the variable names x and y , respectively. Next, all type labels such as PERSON and FOOD are converted to unary predicates with the same name. Conceptual relations such as AGENT, OBJECT, and INSTRUMENT are converted to predicates with as many arguments as there are arcs connected to the relation. Concept referents such as Joe and soup become FOPL constants. Concepts with extended referents such as V map to the universal quantifier V. Generic concepts with no quantifier in the referent field have an existential quantifier, $\exists$, placed before the formula for each variable, and conjunction symbols, &, are placed between the predicates.

As an example, one could convert the sentence "Every car has an engine" from its conceptual graph representation given by

$$[\text{CAR:V}] \rightarrow (\text{PART}) \rightarrow (\text{ENGINE})$$

to its equivalent FOPL representation. Using the rules outlined above, the equivalent FOPL representation derived is just

$$\forall x \exists y (CAR(x) \rightarrow (ENGINE(y) \& PART(x,y)))$$

Mapping the other way, that is from FOPL statements to conceptual graphs, begins by putting the FOPL formula into prenex normal form, and converting all logical connectives to negation and conjunction. Next, every occurrence of universal quantification $\forall x$ is replaced with the equivalent form $\neg \exists \neg x$ (in graph notation this is $\neg [x]$ with the subsequent addition of balancing brackets $]]$ to close off the expression). Every variable x and every occurrence of $\exists x$ is then replaced with the most general type concept denoted as $[T:*x]$. And finally, every n -ary predicate symbol is replaced with an n -ary concept relation whose i th arc is attached to the concept in the i th argument place in the predicate.

Implication in a conceptual graph can be represented with negation and conjunction. For example, the FOPL equivalent of $P \rightarrow Q$ can be written as $\neg [P \neg [Q]]$ (recall that $P \rightarrow Q = (\neg P \vee Q) = \neg (P \& \neg Q)$). In this expression, $\neg$ is read as if and the nested $\neg$ is read as then. More generally, we write the implication as $\neg [*p] [*q]$ where $*p$ and $*q$ are themselves any conceptual graph.

Inference can be accomplished by modifying and combining graphs through the use of operators and basic graph inference rules. Four useful graph formation operators are copy, restrict, join, and simplify. These operators are defined as follows.

Copy. Produces a duplicate copy of a CG.

Restrict. Modifies a graph by replacing a type label of a concept with a subtype or a specialization from generic to individual by inserting a referent of the same concept type.

Join. Combines two identical graphs C_1 and C_2 by attaching all relation arcs from C_2 to C_1 and then erasing C_2 .

Simplify. Eliminates one of two identical relations in a conceptual graph when all connecting arcs are also the same.

As an example of the use of the formation rules, consider the sentence "Tweety ate a fat worm." This sentence can be broken down into five basic or canonical conceptual graphs corresponding to the five words in the sentence.

Tweety:	[BIRD:tweety]
ate:	[Animal] $\leftarrow$ [AGENT] $\leftarrow$ [ATE] $\rightarrow$ [Patient] $\rightarrow$ [ENTITY]
a:	[T: *]
fat:	[FAT] $\leftarrow$ [ATTRIBUTE] $\leftarrow$ [PHYSICAL-OBJECT]
worm:	[WORM]

The $[T: *]$ signifies that something of an unspecified type exists (T is the most general type of all concepts).

From these basic graphs a single conceptual graph can be constructed using the formation operators. First, the subgraph from "a fat worm" is constructed by *restricting* PHYSICAL-OBJECT in the fat graph to WORM and then *joining* it to the graph for worm to get $[FAT] \leftarrow [ATTRIBUTE] \leftarrow [WORM]$. Next, ENTITY in the graph for ate is *restricted* to WORM and *joined* to the graph just completed. This gives

$$[ANIMAL] \leftarrow (AGENT) \leftarrow [ATE] \rightarrow (PATIENT) \rightarrow [WORM] \leftarrow (ATTRIBUTE) \rightarrow [FAT].$$

The final conceptual graph is obtained by *restricting* ANIMAL to BIRD with referent Tweety, *joining* the graphs and labeling the whole graph with PAST (for past tense).

$$(PAST) \rightarrow ([BIRD: tweety] \leftarrow (AGENT) \leftarrow [EAT] \rightarrow (PATIENT) \rightarrow [WORM] \rightarrow (ATTRIBUTE) \rightarrow [FAT])$$

In forming the above graph, concept specialization occurred (e.g., when restriction took place as in PHYSICAL-OBJECT to WORM). Thus, the formation rules and their inverses provide one method of inference. When rules for handling negation and other basic inference rules are combined with the formation rules, a complete inference system is obtained. This system is truth preserving. The inference rules needed which are the equivalent of those in a PL system are defined as follows.

Erasure. Any conceptual graph enclosed by an even number of negations may be erased.

Insertion. Any conceptual graph may be inserted into another graph context which is enclosed by an odd number of negations.

Iteration. A copy of any conceptual graph C may be inserted into a graph context in which C occurs or in which C is dominated by another concept.

Deiteration. Any conceptual graph which could be the result of iteration may be erased from a conceptual graph context.

Double Negation. A double negation may be erased or drawn before any conceptual graph or set of graphs.

As an example of some of the above rules, any graph w may be inserted in the implication $\neg[u \neg[v]]$ to derive $\neg[u w \neg[v]]$. Any graph w may be erased from the consequent of an implication $\neg[u \neg[v w]]$ to derive $\neg[u \neg[v]]$. Any graph w may be erased from the antecedent of an implication only if it has been independently asserted. Thus if u and $\neg[u v \neg[w]]$ then derive $\neg[v \neg[w]]$.

Note that deiteration and double negation are equivalent to modus ponens, that is, given p and $\neg[p \neg[q]]$, deiteration permits erasure of p inside the first bracket to get $\neg[\neg[q]]$. Double negation then permits erasure of $\neg$ to obtain the final result q .

Other inference methods including inheritance (if all A have property P and all B are A , all B have property P) and default reasoning are also possible with conceptual graphs. The implementation of modal logic formalisms with these graphs is possible by using concepts such as possible and necessary. Heuristic reasoning can be accomplished within the theory of conceptual graphs.

In summary, conceptual graphs offer the means to represent natural language statements accurately and to perform many forms of inference found in common sense reasoning.

7.3 FRAME STRUCTURES

Frames were first introduced by Marvin Minsky (1975) as a data structure to represent a mental model of a stereotypical situation such as driving a car, attending a meeting, or eating in a restaurant. Knowledge about an object or event is stored together in memory as a unit. Then, when a new situation is encountered, an appropriate frame is selected from memory for use in reasoning about the situation.

Frames are general record-like structures which consist of a collection of slots and slot values. The slots may be of any size and type. Slots typically have names and values or subfields called facets. Facets may also have names and any number of values. An example of a simple frame for Bob is depicted in Figure 7.6 and a general frame template structure is illustrated in Figure 7.7.

From Figure 7.7 it will be seen that a frame may have any number of slots, and a slot may have any number of facets, each with any number of values. This gives a very general framework from which to build a variety of knowledge structures.

The slots in a frame specify general or specific characteristics of the entity for which the frame represents, and sometimes they include instructions on how to apply or use the slot values. Typically, a slot contains information such as attribute value pairs, default values, conditions for filling a slot, pointers to other related frames, and procedures that are activated when needed for different purposes. For example, the Ford frame illustrated in Figure 7.8 has attribute-value slots (COLOR: silver, MODEL: 4-door, and the like), a slot which takes default values for GAS-MILEAGE, and a slot with an attached *if-needed* procedure.

```
(bob
  (PROFESSION (VALUE professor))
  (AGE (VALUE 42))
  (WIFE (VALUE sandy))
  (CHILDREN (VALUE sue joe))
  (ADDRESS (STREET (VALUE 100 elm))
            (CITY (VALUE dallas))
            (STATE (VALUE tx))
            (ZIP (VALUE 75000))))
```

Figure 7.6 A simple instantiated person frame.

```

(<frame name>
  (<slot1> (<facet1><value1>...<valuek1>)
    (<facet2><value1>...<valuek2>))

  (< slot2> (<facet1><value1>...<valuekm>))

```

Figure 7.7 A general frame structure.

The value *fget* in the GAS-MILEAGE slot is a function call to fetch a default value from another frame such as the general car frame for which Ford is a-kind-of (AKO). When the value of this slot is evaluated, the *fget* function is activated. When *fget* finds no value for gas mileage it recursively looks for a value from ancestor frames until a value is found.

The if-needed value in the Range slot is a procedure name that, when called, computes the driving range of the Ford as a function of gas mileage and fuel capacity. Slots with attached procedures such as *fget* and if-needed are called procedural attachments or demons. They are done automatically when a value is needed but not provided for in a slot. Other types of demons include *if-added* and *if-removed* procedures. They would be triggered, for example, when a value is added or removed from a slot and other actions are needed such as updating slots in other dependent frames.

Like associative networks, frames are usually linked together in a network through the use of special pointers such as the AKO pointer in Figure 7.8. Hierarchies of frames are typical for many systems where entities have supertype-subtype or generic-instance relationships. Such networks make it possible to easily implement property inheritance and default reasoning as depicted in Figure 7.8. This is illustrated

```

(ford (AKO (VALUE car))
  (COLOR (VALUE silver))
  (MODEL (VALUE 4-door))
  (GAS-MILEAGE (DEFAULT fget))
  (RANGE (VALUE if-needed))
  (WEIGHT (VALUE 2800))
  (FUEL-CAPACITY (VALUE 18))

```

Figure 7.8 A Ford frame with various slot types.

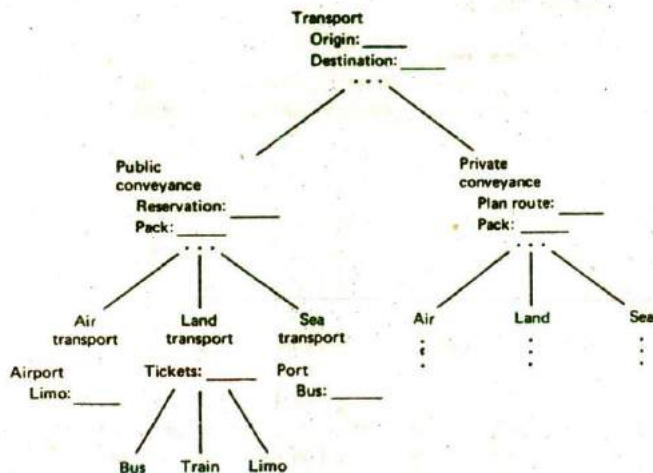


Figure 7.9 Network of frames for transportation methods.

in the network of frames which represents various forms of transportation for people (Figure 7.9).

Frame-Based Representation Languages

Frame representations have become popular enough that special high level frame-based representation languages have been developed. Most of these languages use LISP as the host language. They typically have functions to create, access, modify, update, and display frames. For example, a function which defines a frame might be called with

```
(fdefine f-name <parents><slots>)
```

where `fdefine` is a frame definition function, `f-name` is the name assigned to the new frame, `<parents>` is a list of all parent frames to which the new frame is linked, and `<slots>` is a list of slot names and initial values. Using the function `fdefine` to create a train frame we might provide the following details.

```
(fdefine general-train land-transport
  (type (VALUE passenger))
  (class (VALUE first-class second-class sleeper))
  (food (restaurant (VALUE hot-meals))
        (fast-food (VALUE cold-snacks))
        ...
        ...
        ...))
```

A few other functions typically provided in a frame language include

(fget f-name slot-name facet-name)	:returns data from :specified location
(fslots f-name)	:returns names of :slots
(ffacets f-name slot-name)	:returns names of :facets
(fput f-name slot-name facet-name)	:adds data to a :specified location
(fremove f-name slot-name facet-name)	:removes data from :specified location

It is also possible to represent frame-like structures using object-oriented programming extensions to LISP languages such as FLAVORS (described in Chapter 8).

7.4 CONCEPTUAL DEPENDENCIES AND SCRIPTS

Scripts are another structured representation scheme introduced by Roger Schank (1977). They are used to represent sequences of commonly occurring events. They were originally developed to capture the meanings of stories or to "understand" natural language text. In that respect they are like a script for a play.

A script is a predefined frame-like structure which contains expectations, inferences, and other knowledge that is relevant to a stereotypical situation. Scripts are constructed using basic primitive concepts and rules of formation somewhat like the conceptual graphs described in Section 7.2. Before proceeding with a description of the script, we describe the primitives and related rules used in building them. They are known as conceptual dependencies (not to be confused with conceptual graphs).

Conceptual Dependencies

Conceptual dependency (CD) theory is based on the use of a limited number of primitive concepts and rules of formation to represent any natural language statement. The theory states that different sentences which have the same meaning should have the same unique CD representation. Furthermore, representations for any sentence should be unambiguous as the speaker intends, even though it may have syntactic ambiguity, as in "I saw the Golden Gate Bridge flying into San Francisco this afternoon." It is the contention that any concept, like dreaming, thinking, bellowing, or scheming can be described in terms of these primitives.

In CD theory five different types of ontological (state of being) building blocks are distinguished. Each of these types, in turn, has several subtypes. The types are made up of entities, actions, conceptual cases, conceptual dependencies, and conceptual tenses.

ENTITIES

Picture producers (PP) are actors or physical objects (including human memory) that perform different acts.

Picture aiders (PA) are supporting properties or attributes of producers.

ACTIONS

Primitive actions (ACTs) as listed in Figure 7.10.

Action aiders (AA) are properties or attributes of primitive actions.

Primitive actions	Intended meaning
ATRANS	Transfer of an abstract entity
ATTEND	Focusing attention on an object
CONC	To think about something
EXPEL	Expulsion of anything from the body
GRASP	Grasping or holding an object tightly
INGEST	Ingesting something
MBUILD	Building on information
MOVE	Moving a part of the body
MTRANS	Transfer of mental information
PROPEL	Application of force
PTRANS	Physical transfer from one location to another
SPEAK	Emitting a sound

Figure 7.10 Conceptual dependency primitive actions.

CONCEPTUAL CASES (ALL ACTIONS INVOLVE ONE OR MORE OF THESE)

Objective Case

Directive Case

Instrumental Case

Recipient Case

CONCEPTUAL DEPENDENCIES

Semantic rules for the formation of dependency structures such as the relationship between an actor and an event or between a primitive action and an instrument, (see Figure 7.11).

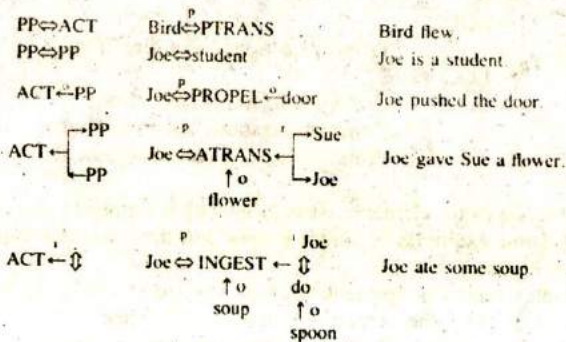


Figure 7.11 Some typical conceptual dependency structures.

CONCEPTUAL TENSES (TIME OF ACTION OR STATE OF BEING)

Conditional (c)

Continuing (k)

Finished Transition (tf)

Future (f)

Interrogative (?)

Negative (/)

Past (p)

Present (nil)

Start Transition (ts)

Timeless (delta)

Transition (t)

Conceptual structures in the form of a graph are used to represent the meanings of different English (or other language) sentences. The graphs are constructed from elementary structures in accordance with basic syntax rules. Some of the basic concept rules are as follows.

PP	ACT	Some picture producers perform primitive actions.
PP	PA	Picture producers have attributes.
ACT ← O — PP		Primitive actions have objects.
ACT ← R —	PP	Primitive actions have recipients.
	pp	
ACT ← D —	LOC	Primitive actions have directions.
	LOC	
T		Conceptualizations have times.
LOC		Conceptualizations have locations.
ACT ← I — ∩		Primitive actions have instruments.

Using these syntactic elements, structures which represent any sentence can be constructed. Some examples of simple graphs and their corresponding sentences are illustrated in Figure 7.11.

More complex sentence representations are constructed from the basic building blocks given above. Note the similarities between CD theory and the conceptual graphs of the previous section. Both have primitive concepts and relations defined, and both have a syntax for graphical representation. Conceptual graphs differ from CDs mainly in that the conceptual graph is logic based, whereas CD theory is mainly concerned with the semantics of events. We now turn to the event representation structure which uses CDs, the script.

Scripts

Scripts are frame-like structures used to represent commonly occurring experiences such as going to the movies, shopping in a supermarket, eating in a restaurant, or visiting a dentist. Like a script for a play, the script structure is described in terms

SCRIPT-NAME:	food market
TRACK	: supermarket
ROLES	: shopper deli attendant seafood attendant checkout clerk sacking clerk other shoppers
ENTRY	
CONDITIONS	: shopper needs groceries food market open
PROPS	: shopping cart display aisles market items checkout stands cashier money

SCENE1	: Enter Market shopper PTRANS shopper into market shopper PTRANS shopping-cart to shopper
SCENE2	: Shop For Items shopper MOVE shopper through aisles shopper ATTEND eyes to display items shopper PTRANS items to shopping cart
SCENE3	: Check Out shopper MOVE shopper to checkout stand shopper WAIT shopper turn shopper ATTEND eyes to charges shopper ATRANS money to cashier sacker ATRANS bags to shopper
SCENE4	: Exit Market shopper PTRANS shopper to exit-market

RESULTS	: shopper has less money shopper has grocery items market has less grocery items market has more money
----------------	---

Figure 7.12 A supermarket script structure.

of actors, roles, props, and scenes. Slots in a script which correspond to parts of the event are filled with CD primitives as defined above. An example of a supermarket script is illustrated in Figure 7.12. This script has four scenes which correspond to the main events which commonly occur in a supermarket shopping experience.

Reasoning with Scripts

Since scripts contain knowledge that people use for common every day activities, they can be used to provide an expected scenario for a given situation.

Reasoning in a script begins with the creation of a partially filled script named to meet the current situation. Next, a known script which matches the current situation is recalled from memory. The script name, preconditions, or other key words provide index values with which to search for the appropriate script. Inference is accomplished by filling in slots with inherited and default values that satisfy certain conditions. For example, if it is known that Joe-PTRANS-Joe into a supermarket and Joe-ATRANS-cashier money, it can be inferred that Joe needed groceries, shopped for items, paid the cashier, checked out, and left the market with groceries but with less money than when he entered.

Scripts have now been used in a number of language understanding systems (English as well as other languages) at Yale University by Schank and his colleagues. One such system is SAM (Script Applier Mechanism) which reads and reasons with text to demonstrate an "understanding" of stories (such as car accident stories from newspapers) that were script based. Other programs developed at Yale include PAM, POLITICS, FRUMP, IPP, BORIS, BABEL, and CYRUS. All of these programs deal with reading, planning, explaining, or in some way understanding stories. They all used some form of script representation scheme.

7.5 SUMMARY

In this chapter we have investigated different types of structured knowledge representation methods. We first considered associative networks, a representation based on a structure of linked nodes (concepts) and arcs (relations) connecting the nodes. With these networks we saw how related concepts could be structured into cohesive units and exhibited as a graphical representation.

Next, we looked at conceptual graphs, a structured formalism based on traditional logics and which uses primitive building blocks for concepts and relationships between the concepts. How conceptual graphs and formulas in FOPL could be mapped from one to the other and how inferring with conceptual graphs compared to logical inference were demonstrated.

We then considered frame structures as general methods of representing units of knowledge. Frames are composed of any number of slots, each with any number of facets, and they in turn, contain any number of values. The contents of a slot may be attributes which characterize the frame entity, pointers to related frames,

procedures, or even other subframes. Inference with frames is accomplished through property inheritance, default values, embedded procedures, and the use of heuristics.

Finally, we described a special frame-like structure called a script. Scripts are used to represent stereotypical patterns for commonly occurring events. Conceptual dependency theory provides primitive building blocks for actions and states that occur within a script. Like a play, a script contains actors, roles, props, and scenes which combine to represent a familiar situation. Scripts have been used in a number of programs which read and "understand" language in the form of stories.

EXERCISES

- 7.1** Express the following concepts as an associative network structure with interconnected nodes and labeled arcs.

Company ABC is a software development company. Three departments within the company are Sales, Administration, and Programming. Joe is the manager of Programming. Bill and Sue are programmers. Sue is married to Sam. Sam is an editor for Prentice Hall. They have three children, and they live on Elm street. Sue wears glasses and is five feet four inches tall.

- 7.2 Write LISP expressions which represent the associative network of Problem 7.1.
- using property lists, and
 - using a-lists.

- 7.3** Write PROLOG expressions which represent the associative network of Problem 7.1.

- 7.4** Transform the FOPL statements given below into equivalent conceptual graphs.

- a. $\forall x \text{ NORMAL}(x) \ \& \ \text{GROWN}(x) \rightarrow \text{WALK}(x)$.
 b. $\forall x, y \text{ MARRIED}(x, y) \rightarrow \text{MARRIED}(y, x)$.
 c. $\forall x \text{ HASWINGS}(x) \ \& \ \text{LAYSEGGS}(x) \rightarrow \text{ISBIRD}(x)$.

- 7.5** Transform the following conceptual graphs into equivalent FOPL statements.

- a. [PERSON:sue] ← (AGENT) ← [DRINK]-
(OBJECT) → [FOOD:milk]
(INSTRUMENT) → [GLASS]
b. (PAST) → [[CAMEL:clyde] ← (AGENT) ← [DRINK]- (OBJECT)-
[WATER] → (ATTRIBUTE) → [50-GALLONS]]

- 7.6** The original primitives of conceptual dependency theory developed by Schank fail to represent some important concepts directly. What additional primitives can you discover that would be useful?

- 7.7** Create a movie script similar to the supermarket script of Figure 7.11.

- ### 7.8 What are the main differences between scripts and frame structures?

- 7.9** Express the following sentences as conceptual dependency structures.

- a. Bill is a programmer.
- b. Sam gave Mary a box of candy.
- c. Charlie drove the pickup fast.

- 7.10** Create a frame network for terrestrial motor vehicles (cars, trucks, motorcycles) and give one complete frame in detail for cars which includes the slots for the main component

- parts, their attributes, and relations between parts. Include an as-needed slot for the gas of each type mileage.
- 7.11 Write a LISP program to create a frame data structure which represents the car frame of Problem 7.10.
- 7.12 Compare the inference process using frames to that of inference in FOPL. Give examples of both.

8

Object-Oriented Representations

The previous chapter marked a departure from the approaches to knowledge representation of earlier chapters in that the methods there focused on adding structure to the knowledge. Structure was added by linking concepts through relations and clustering together all related knowledge about an object. In some cases, as with frames, procedures related to the knowledge were also attached to the knowledge cluster. The approach in grouping knowledge and related procedures together into a cohesive unit is carried even further with object-oriented systems which we examine in some detail in this chapter.

8.1 INTRODUCTION

Grouping related knowledge together in AI systems gains some of the same cognitive advantages realized in the human brain. The knowledge required for a given cognitive task is usually quite limited in domain and scope. Therefore, access and processing can be made more efficient by grouping or partitioning related knowledge together as an unit. We saw how this notion was implemented with linked frames in the previous chapter. We shall see in this chapter, that object-oriented systems share a number of similarities with the frame implementations.

In procedural programming languages such as Pascal or FORTRAN, a program

consists of a procedural part and a data part. The procedural part consists of the set of program instructions, and the data part, the numbers and character strings that are manipulated by the instructions. Programs typically contain several modules of instructions that perform computations on the same data set. When some change is made to the format of the data, every module that uses it must then be modified to accommodate the newly revised format. This places a heavy burden on the software maintenance process and makes these types of programs more prone to errors.

In an object-oriented system (OOS) the emphasis between data and procedures is reversed. Data becomes the primary object and procedures are secondary. For example, everything in the universe of an OOS is an object, and objects are inaccessible to outside procedures. This form of structuring is sometimes called encapsulation or data hiding. It is a well known system design principle used to make systems more modular and robust. With encapsulation, objects are associated with their own procedures and, as such, are responsible for their own actions. Thus, when some change is required in the data or procedure, only the changed object need be modified. Other objects are not affected and therefore require no modifications.

In object-oriented systems there is a simplicity in structure because almost everything is an object. For example, a car can be regarded as an object consisting of many interacting components or subobjects: an engine, electrical system, fuel system, drive train, controls, and so on. To model such a system using an object-oriented approach requires that all parts be declared as objects, each one characterized by its own attributes and its own operational behavior. Even a simple windshield wiper would be described as an object with given attributes and operations. As such, it might be described as the structure presented in Figure 8.1.

This object has a name, a class characterization, several distinguishing attributes, and a set of operations. Since all characteristics of the wiper object, including its operations, are contained within a single entity, only this entity needs changing when some design change is made to this part of the car. Other objects that interact with it are not affected, provided the communication procedures between the objects were not changed. To initiate the task of cleaning moisture from the windshield requires only that a message be sent to the object. This message can remain the

OBJECT NAME	left wiper
AKO	wiper
ATTRIBUTES	made of rubber and metal length: 14 inches color: black and silver location: lower left windshield function: rub moisture from windshield
OPERATIONS	turn-on switch: move in arc on windshield repeating clockwise then counter-clockwise turn-off switch: move to home position

Figure 8.1 Object description for a windshield wiper.

same even though the structure of the wiper or its mode of operation may have changed.

Because there is more than one wiper, each with similar attributes and operations, some savings in memory and procedures can be realized by creating a generic class which has all the characteristics which are common to the left, right, and rear wipers. The three instances retain some characteristics unique to themselves, but they inherit common attributes and operations from the more general wiper class.

The object paradigm described above seems to model real-world systems more closely than the procedural programming models where objects (data) and procedures are separated. In object-oriented systems, objects become individual, self-contained units that exhibit a certain behavior of their own and interact with other objects only through the passing of messages. Tasks get performed when a message is sent to an object that can perform the task. All the details of the task are rightfully hidden from other objects. For example, when your car needs repairing, you send a message to the repair shop. The repair shop, in turn, may need parts from one or more manufacturers for which they must send messages. When the car has been repaired, the repair shop sends you a message informing you of that fact.

In having the shop repair your car, you probably are not interested in all the details related to the repair, the fact that messages were sent to other organizations for parts, that they were obtained by Federal Express, and that certain detailed procedures were taken to complete the repair process. Your primary concern is that the repair operation was properly completed and your car returned in working order. The need to model operational behavior such as this has prompted the development of object-oriented systems.

2 OVERVIEW OF OBJECT-ORIENTED SYSTEMS

The basic idea behind an OOS is the notion of classes of objects interacting with each other to accomplish some set of tasks. The objects have well-defined behaviors. They interact with each other through the use of messages. When a task needs to be performed, an object is passed a message which specifies the task requirements. The receiving object then takes appropriate action in response to the message and responds by returning a message to the sender. In performing the required task, the receiver may need assistance from other objects, thereby prompting further messages to be sent.

These ideas are illustrated in Figure 8.2 which depicts the simulation of a seaport facility. Ocean ships arrive for docking, unloading, loading, and departing. When the facilities (tugboats, berths, and loading and unloading equipment and crews) are busy, arriving ships must queue and wait at sea until space and other facilities are available. The harbor master coordinates the arrivals and departures by assigning tugs and other resources to the arriving ships. The objects in this example are, of course, the individual ships, the tugs, the docks, the harbor master,

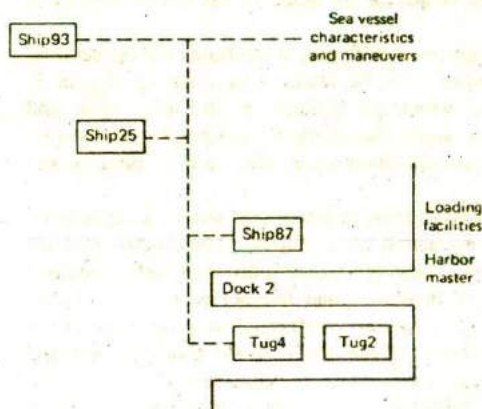


Figure 8.2 Objects communicating to complete a task.

and the cargo handling facilities. Actions are initiated by message passing between these objects. The dashed lines connecting members of the class of sea vessels depict the way common characteristics and operational behaviors are shared by members of the same class (they all have a coordinate position, a maximum cruising speed, cargo capacity, and so on).

Tasks are performed when a message is sent from one object to another. For example, the harbor master may send a message to a tug to provide assistance to ship 87 in deberting from dock 2. This would then result in a sequence of actions from the tug having received the message.

In general, a task may consist of any definable operation, such as changing an object's position, loading cargo, manipulating a character string, or popping up a prompt window. A complete program would then be a sequence of the basic tasks such as the simulated movement of ships into and out of the seaport after discharging and taking on cargo.

8.3 OBJECTS, CLASSES, MESSAGES, AND METHODS

In this section we present definitions for the basic concepts that make up an OOS: the object, message, class, methods, and class hierarchies. There are probably as many as fifty different OOS languages, and the examples presented in this section may not comply exactly with any one in particular. The examples are representative of all OOS however, and are based mainly on material from the Smalltalk family, including Smalltalk 80 (Goldberg and Robson, 1983, and Kaehler and Patterson, 1986), Smalltalk/V (Digitalk, Inc., 1986), and Little Smalltalk (Budd, 1987). Specialized OOS languages are considered in Section 8.5.

Objects

Objects are the basic building blocks in object-oriented systems. All entities except parts of a message, comments, and certain punctuation symbols are objects. An object consists of a limited amount of memory which contains other objects (data and procedures). They are encapsulated together, as a unit and are accessible to that object only. Examples of objects are numbers such as 5, 31, 6.213, strings like 'this is a string,' arrays such as #(23 'a string' 311 (3 4 5)), the Turtle (a global graphics object originally used in LOGO), a windshield wiper as described above, a ship, and so on. Objects are characterized by attributes and by the way they behave when messages are sent to them. All objects belong to some class. They are created by declaring them as instances of an existing class and instantiating instance variables. The class to which an object belongs can be determined by sending it the message "class."

Messages

Actions are performed in an OOS by sending messages to an object. This corresponds to a function or procedure call in other languages. The messages are formatted strings composed of three parts: a receiver object, a message selector, and a sequence of zero or more arguments. The format of a message is given as

`<object><selector><arg1 arg2 ...>`

The object identifies the receiver of the message. This field may contain an object item or another message which evaluates to an object. The selector is a procedure name. It specifies what action is required from the object. The arguments are objects used by the receiver object to complete some desired task. Messages may also be given in place of an argument since a message always elicits an object as a response.

When an object receives a valid message, it responds by taking appropriate actions (such as executing a procedure or sending messages to other objects) and then returning a result. For example, the message `9 - 5` causes the receiver object 9 to respond to the selector `-` by subtracting 5 from 9 and returning the object 4.

There are three types of messages: unary, binary, and keyword (*n*-ary). All three types parse from left to right, but parentheses may be used to determine the order of interpretation. A unary message requires no arguments. For example, each of the following are unary messages:

```
5 sign
10 factorial
'once upon a time' size
#(a b c d) reversed
68 asCharacter
```


In each of these examples, the first item in the message is the receiver object, and the second item the selector. The first example returns the integer +1 to signify a positive algebraic sign for the number 5. The second example returns 3628800 the factorial value of the integer 10. The third example returns 16, the length of the string. The fourth returns the array $\#(d\ c\ b\ a)$, and the fifth returns D , the ASCII character equivalent of 68.

Binary messages take one argument. Arithmetic operations are typical of binary messages, where the first operand is the receiver, the selector is the arithmetic operation to be performed, and the second operand is the argument. Examples of binary messages are

10 + 32	"an addition message"
13 - 9	"a subtraction message"
22 * 7	"multiplication message"
54 / 2	"rational division message"
$\#(a\ b\ c), \#(d\ e\ f)$	"the comma concatenates two arrays"
7 < 9	"relational test message"
7 (a 12	"an x-y coordinate point reference"

Comments may be placed anywhere within an OOS program using the double quotation marks as seen in the above examples. Note that the last three examples are nonarithmetic binary messages. They result in the combining of two arrays into one, a boolean relational test, and the creation of a graphics coordinate point at column 7, row 12, respectively.

The third and most general type of message is the keyword message. These messages have selectors which consist of one or more keyword identifiers, where each is followed by a colon and an argument. The argument can be an object or any message, but if it is another keyword message, it must be enclosed in parentheses to avoid ambiguity. Examples of keyword messages are

5 between: 4 and: 10	"a Boolean test"
'aecdb' copyFrom: 2 to: 5	"copies position 2 of the string to position 5"
$\#(a\ b\ c\ x)$ at: 4 put: $\#(d\ e)$	"the elements of $\#(d\ e)$ replace x in the array"
'texas' size between: 2 + 2 and: 4 factorial	
set1 add: (i + 1)	"add new element to set 1"

The last two examples above contain messages within messages, while the last example has a message delimited with parentheses. In executing a message

without parentheses, the execution proceeds left to right with unary messages taking precedence followed by binary, and then keyword. Therefore, the messages 'texas' size and 4 factorial are completed before the keyword part between:and: in the last example above.

Methods

Procedures are called methods. They determine the behavior of an object when a message is sent to the object. Methods are the algorithms or sequence of instructions executed by an object. For example, in order to respond to the message $5 + 7$, the object 5 must initiate a method to find the sum of the integer numbers 5 and 7. On completion of the operation, the method returns the object 12 to the sending object.

Methods are defined much like procedures in other programming languages using the constructs and syntax of the given OOS. The constructs used to build higher level methods are defined in terms of a number of primitive operations and basic methods provided as part of the OOS. The primitives of an OOS are coded in some host language such as an assembler language or C. For example, the operation for integer addition used in some versions of Smalltalk would be written as

```
+ aNumber
^ <SameTypeOfObject self aNumber>
  ifTrue: [<IntegerAddition self aNumber>]
  ifFalse: [super + aNumber]
```

The name of this method is + and the argument is an object of type aNumber. The primitive operation SameTypeOfObject tests whether the two object arguments are of the same type (instances of the same class). The variable self is a temporary variable of an instance of the class it belongs to, Integer. If the two objects are of the same type, the primitive IntegerAddition in the ifTrue block of code is executed and the sum returned. Otherwise, a search for an appropriate method is made by checking the superclass of this class (the class Number). The up-arrow ^ signifies the quantity to be returned by the method.

A typical OOS may have as many as a few hundred predefined primitives and basic methods combined. We will see examples of some typical methods in the next section.

Classes and Hierarchies

A class is a general object that defines a set of individual (instance) objects which share common characteristics. For example, the class of rabbits contains many individual rabbit objects, each with four legs, long ears, whiskers, and short bushy tails. The class of natural numbers contains many instance objects such as 43, 91, 2, All objects are instances of some class and classes are subclasses of some higher

class, except for a most general root class. The root class for an OOS is the class named *Object*.

Classes can often be divided into subclasses or merged into superclasses. The class of fruit can be divided into citrus and noncitrus, both of which can be further divided. Fruit is part of the superclass of plant-grown foods which in turn is part of the class of all foods. Classes permit the formation of hierarchies of objects which can be depicted as a tree or taxonomic structure as illustrated in Figure 8.3.

Objects belonging to the same class have the same variables and the same methods. They also respond to the same set of messages called the *protocol* of the class. Each class in a hierarchy inherits the variables and methods of all of its parents or superclasses of the class.

When a message is sent to an object, a check is first made to see if the methods for the object itself or its immediate class can perform the required task. If not, the methods of the nearest superclass are checked. If they are not adequate, the search process continues up the hierarchy recursively until methods have been found or the end of a chain has been reached. If the required methods are not found, an error message is printed.

Some OOSs permit classes to have two or more direct superclasses (Stefik and Bobrow, 1986). For example, *StereoSystem* may have superclasses of *Appliances*, *LuxuryGoods*, and *FragileCommodity*. As such, a stereo object may inherit characteristics and methods from all three superclasses. When this is the case, an inheritance precedence must be defined among the superclasses. One approach would be to try the leftmost superclass path in the hierarchy first. If applicable methods are not found up this path, the next leftmost path is taken. This process continues progressively shifting to the right until a method is found or failure occurs.

An OOS will have many predefined classes. For example, a few of the classes for the Smalltalk family and their hierarchical structure are depicted in Figure 8.4.

Each of the classes depicted in Figure 8.4 has a number of methods that respond to the protocol for the class. A class may also inherit methods from a superclass. For example, all classes inherit the method "`== anObject`" which answers true if the receiver and *anObject* are the same, and answers false otherwise.

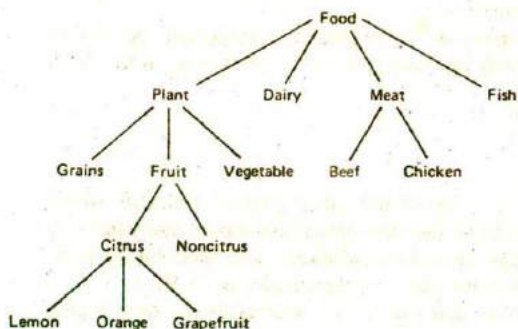


Figure 8.3 A class hierarchy of foods.

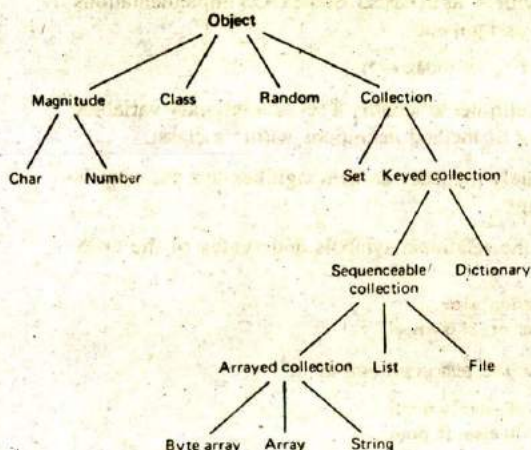


Figure 8.4 Partial hierarchy of predefined OOS classes.

Polymorphism is the capability for different objects to respond to the same message protocols but exhibit their own unique behaviors in response. For example, a message containing the selector `moveForward` could invoke the forward movement of a ship as well as advancing a piece in a game such as checkers. Both classes use the same message template but respond differently. The only requirement is that the message protocol for the two classes be implemented as required for the given class.

8.4 SIMULATION EXAMPLE USING AN OOS PROGRAM

In this section, we present a simple example of an OOS program to demonstrate some of the features defined above. The example is a program to simulate the seaport operations described in Section 8.2. Before we begin however, we define some additional syntax and operations used in an OOS.

An OOS will have most of the basic programming constructs of a procedural language such as Pascal, including arithmetic and string manipulation operations, assignment and conditional statements, logical expressions, iteration, and so on. A few examples will help to illustrate some of the basic constructs and syntactical conventions.

1. A period is used as a statement separator, not as a terminator.

2. A block object is a sequence of OOS statements (messages) enclosed within square brackets. A block is like an in-line procedure that, when evaluated, returns the last value executed within the block. Blocks are instances of the `Block` class that execute when sent the message `value`, `value:` or `value: value:`, depending on whether the block has zero, one, or two arguments.

3. Variable assignment is made with `:=` as in Pascal. Some OOS implementations use the back arrow `←` like Algol for assignment

```
index := 1 (or index ← 1).
```

4. The vertical bar is used as a delimiter to identify a set of temporary variables used in methods as well as a separator of method definitions within a class.

5. An up arrow which immediately precedes an item signifies that the item is to be returned in response to a message.

6. Boolean relational tests use the relational symbols and syntax of the OOS such as

```
5 < 'string' size
('camel' at: 3) isVowel.
```

7. Conditional statements follow the test condition as in

```
a < b ifTrue: [a print]
      ifFalse: [b print].
```

8. Typical logical expressions are given by

```
a >= b and: [c <= d]
x isDigit or: [$F <= y and: [y <= $L]]
```

The dollar sign preceding a character identifies character objects.

9. Typical iteration constructs are `whileTrue`, `whileFalse`, `timesRepeat`, and `do`.

```
i := 1.
[i print. i := i + 2. i <= 10] whileTrue.
```

```
1 to: 10 do [|:j] array at: j] print).
```

```
paths := 6.
paths timesRepeat [ship move: 100; turn: 360//paths].
```

10. The variable `self` in a method refers to the receiver object of the message that invokes the method. The variable `super` is used in a method to invoke a search for a method in an object's superclass.

In addition to the above examples, an OOS will have many special methods for the definition of classes and objects and for the definition of class behaviors and class related tasks.

Simulation of Seaport Operations

An event driven simulation of the seaport operation described in Section 8.2 is a computed sequence of the events of interest which occur at discrete time points.

This system would have as a minimum the three events: (1) cargo ship arrivals, (2) ship berthing operations, and (3) cargo transfer and ship departures. These events are symbolized by the following expressions which will be used in our program.

shipArrival
shipDocking
shipDeparture

In the interest of clarity, several simplifying assumptions are made for the simulation problem. First, we limit our objects of interest to three classes, namely the class of ships (three types of cargo ships), the group of entities which make up the harbor operations (tugs, docks, cranes, crews, and the like) treated collectively as one object class called HarborOperations, and the class called Simulator. The Simulator class is defined to permit separate simulation runs, that is, separate instances of Simulator.

Second, we assume that ships arriving to find all berths full depart immediately from the system. Ships arriving when at least one of the eight berths is available are scheduled for docking and cargo transfer. Once a ship has been docked, its departure is then scheduled.

To add some realistic randomness to the operation, the time between ship arrivals is assumed to be exponentially distributed with a mean value of 3 time units. The time to dock is assumed to be uniformly distributed with a range of 0.5 to 2.0 time units, and the time to transfer cargo is assumed to be exponentially distributed with a mean of 14 time units. Finally, to simulate three different types of ships, a newly arriving ship is randomly assigned a cargo weight of 10, 20, or 30 thousand tons from an empirical distribution with probabilities 0.2, 0.5, and 0.3, respectively.

The three types of simulated events may occur at any discrete time point, and they may even occur concurrently. To manage these events, we require a system clock to assign scheduled event times and a data structure in which to record all pending events. At any time during the run, the pending event list could include a scheduled ship arrival at current time t_{now} plus some time increment t_1 , the berthing of a ship at time $t_{\text{now}} + t_2$, and the departures of one or more ships at $t_{\text{now}} + t_3$, $t_{\text{now}} + t_4$, and so on. Scheduled events are removed from the list of pending events in the order of smallest time value first.

Pending events are held in a dictionary data structure which contains index-value pairs of objects. And, since multiple events may occur at the same time points, we use a set to hold all events indexed by the same time value. Thus, pending events will be stored in a dictionary of indexed sets with each set containing one or more of the basic events.

Messages required to access sets and dictionary objects (collectively referred to as collections) are needed in the program. The messages and the corresponding actions they elicit are as follows.

MESSAGE	RESULTING ACTION
add:	adds an element to the receiver collection, like a list
at:	returns the item in the dictionary whose key matches the argument
at: ifAbsent:	returns the element given by the key in the first argument and evaluates the second argument if no argument exists
at: put:	places the second argument into the receiver collection under the key given by the first argument
first	returns the first element from a Set.
includesKey:	returns true if the key is valid for the receiver
isEmpty	returns true if the receiver collection contains no elements
keysDo:	evaluates each key element of the one argument block which follows according to the procedure given in the block
remove:	removes the argument object from the receiver collection
removeKey:	removes the object with the given key from the receiver collection

For output from the simulation, we print the arrival time of each ship, indicating whether it docks or not, each ship departure, and the total cargo transferred at the end of the run.

With the above preliminaries, we now define the three classes and their corresponding methods. We begin with the class Simulator which is the most complicated. To define a class, the word Class is given followed by the class name and (optionally) by the class's immediate superclass; if no superclass is given, the default class Object is assumed. This is followed by a list of local variables within vertical bar delimiters. The method protocol for the class is defined next, with the methods separated by vertical bars. The message template for each method is given as the first item following the vertical bar. When local variables for a method are needed, they follow the message template, also given within vertical bars.

```

Class Simulator
|currentTime eventsPending|
|
  new
    eventsPending := Dictionary new.
    currentTime := 0
|
  time
    ^currentTime
|
  addEvent: event at: eventTime
    (eventsPending includesKey: eventTime)
    ifTrue: [(eventsPending at: eventTime) add: event]

```

```

        ifFalse: [eventsPending at: eventTime
                  put: (Set new ; add: event)]

    addEvent: event next: delayTime
        self addEvent: event at: currentTime + delayTime

    proceed [minTime eventSet event]
        mintime := 99999.
        eventsPending keysDo:
            [:x| (x < minTime) ifTrue: [minTime := x]].
        currentTime := minTime.
        eventSet := eventsPending at:
            minTime ifAbsent: [nil].
        event := eventSet first.
        eventSet remove: event.
        (eventSet isEmpty)
            ifTrue: [eventsPending removeKey: minTime].
        self processEvent: event

```

The method responding to the message addEvent checks to see if a time value (key) exists in the dictionary. If so, it adds the new event to the set under the key. If the time does not already exist, a new set is created, and the event is added to the set and the set put in eventsPending. The proceed method finds the smallest event time (key) and retrieves and removes the first element of the set located there. If the resultant set is empty, the key for the empty set is removed. A message is then sent to the processEvent object in the class HarborOperations which is defined next.

```

Class HarborOperation : Simulator
|totalCargo arrivalDistribution dockingDistribution
 serviceDistribution remainingBerths|
{
    new
        totalCargo := 0.
        remainingBerths := 8.
        arrivalDistribution := Exponential new: 3.
        shipDistribution := DiscreteProb new: #(0.2 0.5 0.3)
        dockingDistribution := Uniform new: #(0.5 2.0).
        serviceDistribution := Exponential new: 14.
        self scheduleArrival

    scheduleArrival [newShip time]
        newShip := Ship new.
        self addEvent: [self shipArrival: newShip]
            at: (self time + (arrivalDistribution next))

```



```

processEvent: event
    event value.
    ('ship arrived at', self time) print.
    totalCargo := totalCargo + (shipSize * 10).
    self scheduleArrival
|
reportCargo
    ('total cargo transferred', totalCargo) print
|

```

The method new initializes some variables, including the arrival, docking, and service distributions. A sample from a distribution is obtained by sending the distribution the message next. (The programming details for the generation of random samples from all distributions have been omitted in the interest of presenting a more readable example.) The scheduleArrival method sends a message to the Ship class to create a new ship and then adds the event block

```
[self shipArrival: newShip]
```

to the pending event list at the arrival time value. The processEvent method is activated from the proceed method in the Simulator class. It initiates evaluation of the event block stored in the pending list, prints a ship arrival message, and computes the new total for the cargo discharged.

Next, we define the class Ship.

```

Class ship
|shipSize|
{
    new
        shipSize := shipD.stribution next
    |
        shipSize
        ~ shipSize
    |
}

```

With the object classes defined, we can now write the statements for the three object events. The arrival event is initiated from the HarborOperation class. This event then schedules the next operation (docking), which in turn schedules the ship departure event.

```

shipArrival: ship
    (remainingBerths > 0)
    ifTrue: [remainingBerths := remainingBerths - 1.
            self addEvent: [self shipDocking: ship]
                        at: (self time + dockingDistribution next)].
    ifFalse: ['all berths occupied, ship departs' print]

```

```

shipDocking: ship
  totalCargo := totalCargo + shipSize.
  self addEvent: [self shipDepart: ship]
  next: (serviceDistribution next)

shipDepart: ship
  'ship departs after cargo transfer' print.
  remainingBerths := remainingBerths + 1

```

To run the simulation program we simply execute the following statements.

```

Simulator new.
port := HarborOperation new
[port time < 720] whileTrue: [port proceed]

```

Note that the message "port proceed" in the whileTrue block is sent to HarborOperation. Since there is no method proceed in this class, it must be inherited from the Simulator class.

An environment for an OOS will usually include all of the basic primitives, class and method definitions, an editor, a browser, window and mouse facilities, and a graphics output.

8.5 OBJECT-ORIENTED LANGUAGES AND SYSTEMS

In addition to the Smalltalk family of OOS languages, a number of other languages have been developed, including object-oriented extensions to LISP dialects and special purpose languages. Typical of the LISP extensions is the FLAVORS add-ons.

OOS.with Lisp Extensions

In FLAVORS, classes are created with the defflavor form, and methods of the flavor are created with defmethod. An instance of a flavor is created with a make-instance type of function. For example, to create a new flavor (class) of ships with instance variables x-position, y-position, x-velocity, y-velocity, and cargo-capacity, the following expression is evaluated:

```

(defflavor ship (x-position y-position x-velocity
  y-velocity cargo-capacity))

```

Methods for the ship flavor are written in a similar manner with a defmethod, say for the ship's speed, as

```

(defmethod (ship :speed) ()
  (sqrt (+ (* x-velocity x-velocity)
    (* y-velocity y-velocity))))

```


To create an instance of ship one then uses the form

```
(setf ship42 (make-instance 'ship))
```

In addition to user defined methods, three additional predefined methods are available for a flavor. They are used to assign values to instance variables, to initialize variable values, and to get values of the variables. These options are specified in the `defflavor` statement when the flavor is created. For example, to include these optional methods, the following form is used:

```
(defflavor ship (x-position y-position x-velocity
                y-velocity cargo-capacity)
  ()
  :gettable-instance-variables
  :settable-instance-variables
  :inittable-instance-variables)
```

Values for the ship instance variables can now be assigned either with a message or when an instance of the ship is created.

```
(send ship42 :set-cargo-capacity 22.5) or
```

```
(setf ship42 (make-instance 'ship :x-position 5.0
                                :y-position 8.0))
```

Variable assignments can be examined with the `describe` method, one of the base methods provided with the system.

```
(describe ship42)
#<SHIP 1234567>, an object of flavor SHIP,
has instance variable values:
  X-POSITION      5.0
  Y-POSITION      8.0
  X-VELOCITY      unbound
  Y-VELOCITY      unbound
  CARGO-CAPACITY  22.5
```

Default values can also be assigned to instance variables with the `defvar` statement.

```
(defvar *default-x-velocity* 12.0)
```

Thus, unless `x-velocity` is explicitly assigned a value in a `make-instance` or `defflavor` statement, it will be given the default value 12.0.

Flavors are defined hierarchically by including one or more superclasses in the second subform of the `defflavor` statement, thereby permitting flavors to be

"mixed." Inheritance of methods is then achieved much the same as in the Smalltalk case. For example, to create a ship flavor which has two superclass flavors named moving-object and pleasure-craft, the following form would be used:

```
(def flavor ship (x-position y-position passenger-capacity)
  (moving-object pleasure-craft)
  :gettable-instance-variables)
```

If the flavor moving-object has a method for speed, it will be inherited unless a method for speed has been defined explicitly for the ship flavor.

The base flavor for FLAVOR extensions is the vanilla-flavor. This flavor will typically have a number of base methods including a :print-self and describe method as used above. Generic instance variables are also defined for the vanilla-flavor. Through method inheritance and other forms, methods may be combined to give a variety of capabilities for flavors including the execution of some methods just prior to or just after a main method.

Special Purpose OOS Languages

A typical special purpose OOS language is ROSS (for Rand OOS) developed by the Rand Corporation for military battle simulations (Klahr et al, 1980, 1982). This system has been implemented in several dialects of LISP as an interactive simulation system which includes a movie generator and graphics facility. Visual representations can be generated as the simulation is running, and on-the-fly changes can easily be made to a program. It has been used to simulate both air and ground battles.

Messages are sent to objects in ROSS with an "ask" form having the following structure.

```
(ask <object> <message>)
```

For example, to send a message to a fighter-base requesting that a fighter be sent to intercept a penetrator, the following message might be sent:

```
(ask fighter-base1 send fighter2 guided by radar3
  to penetrator2)
```

In response to this message, a method associated with the fighter-base class would be evaluated and appropriate actions would then be initiated through direct computations and embedded message transmissions to other objects.

Object hierarchies which provide variable and method inheritance in much the same way as FLAVORS or Smalltalk can also be created. A simulation program in ROSS would be created in a similar manner to that of the example presented in the previous section, but with a number of basic methods for simulation behaviors

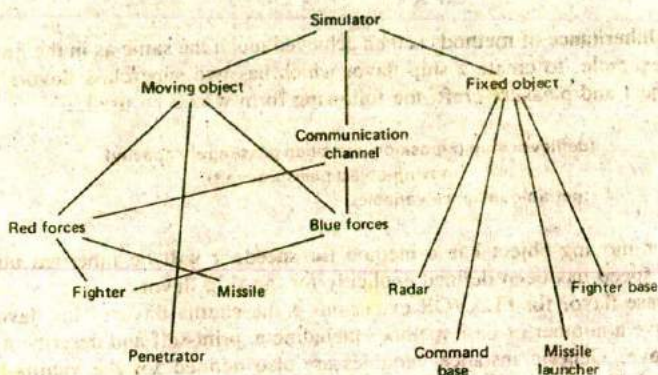


Figure 8.5 An example of a battle class hierarchy.

already predefined. A typical class hierarchy for a battle simulation might be defined as the structure illustrated in Figure 8.5.

8.6 SUMMARY

Object-oriented systems are useful in modeling many real-world situations in which real objects can be put in a one-to-one correspondence with program classes and objects. In an OOS all entities are objects, and objects are protected from other entities. They interact with other objects through an interface which recognizes a set of messages called the protocol for the class. Each class within an OOS will have its own unique behavior which is governed by a set of class methods. Methods and instance variables may be inherited by a class of objects from its parents or superclasses. Multiple inheritance is also supported by many systems.

OOS languages are well suited for certain types of system simulation problems because of the natural way in which OOS programs model real systems. To build a simulation program in an OOS such as Smalltalk, one first defines the object classes required, their hierarchical relationship with each other, and the behaviors of the objects within each class. The events of importance are also defined and the sequence in which they may occur. Message formats for class protocols are then defined, and the specific methods are then coded.

OOS capabilities have been developed for several LISP systems as add-ons such as found in FLAVORS. Special purpose OOS languages have also been developed such as the ROSS system which was developed to provide capabilities not available in other simulation languages.

EXERCISES

- 8.1. Show the order of evaluation for the subexpressions given in the following expression:
 $9/2$ between: $8 + 19 \text{ sqrt}$ and: $4 * 5$
- 8.2. What values will be printed after the following sequences?
- $i := 17$
 $j := [i := i + 1]$
 $i \text{ print}$
 - $j \text{ value print}$ (after the sequence in a above)
 - $i \text{ value print}$ (after the sequence in b above)
- 8.3. What is the class of Class? What is the superclass of Class?
- 8.4. What is the result from typing the following expression?

$$3 + (4 \text{ print}) + 6$$

- 8.5. A bag is like a set except the same item may occur more than once. One way to implement the class Bag is with a dictionary, where the value contained in the dictionary is the number of times the item occurs in the bag. A partial implementation for a bag is given below. Complete the implementation, keeping in mind that instances of dictionary respond to first and next with values and not keys. The current key is accessible, however, if currentKey is used.

```

Class Bag :Collection
| dict count
|
|   new
|   dict:=Dictionary new
|   ... some methods go here. ...
|   first
|   (count:=dict first) isNil ifTrue:[ * nil].
|   count:=count - 1.
|   ^ dict currentKey
|   next
|   [count notNil] whileTrue:
|   [(count>0)
|   ifTrue:[count:=count - 1. ^ dict currentKey]
|   ifFalse:[count:=dict next]].
|   nil
|

```

- 8.6. One method of defining a discrete probability distribution is to provide the actual sample space elements in a collection. A random sample can then be obtained from the collection entries. Produce a class description for a class called SampleSpace which will be used to randomly select points using the following: